

Contents

1	PlusDateApp	6
1.1	Project Information	6
1.2	Links	6
1.3	Elevator Pitch	7
1.4	Evaluation Criteria Checklist	7
1.5	Documentation Navigation	7
2	1. Project Overview	7
2.1	Contents	7
2.2	Executive Summary	8
2.3	Key Highlights	8
3	Problem Statement & Goals	8
3.1	Context	8
3.2	Problem Statement	8
3.2.1	Pain Points	9
3.2.2	Before / After	9
3.3	Business Goals	9
3.4	Objectives & Metrics	10
3.5	Success Criteria	10
3.5.1	Must Have	10
3.5.2	Nice to Have	10
3.6	Non-Goals	11
4	Stakeholders & Users	11
4.1	Target Audience	11
4.2	User Personas	11
4.2.1	Persona 1: Anna, Regular User	11
4.2.2	Persona 2: Mark, Premium-Oriented User	11
4.2.3	Persona 3: Project Maintainer	12
4.3	Stakeholder Map	12
4.3.1	High Influence / High Interest	12
4.3.2	High Influence / Low Interest	12
4.3.3	Low Influence / High Interest	12
4.3.4	Low Influence / Low Interest	12
5	Project Scope	13
5.1	In Scope	13
5.2	Out of Scope	13
5.3	Assumptions	14
5.4	Constraints	14
5.5	Dependencies	15
6	Features & Requirements	15
6.1	Epics Overview	15
6.2	User Stories	15
6.2.1	Epic 1: Identity and Onboarding	15
6.2.2	Epic 2: Profile and Preferences	16
6.2.3	Epic 3: Discovery and Matching	17
6.2.4	Epic 4: Communication	17
6.2.5	Epic 5: Safety and Monetization	18
6.3	Use Case Diagram	18
6.4	Non-Functional Requirements	18

6.4.1	Performance	18
6.4.2	Security	18
6.4.3	Accessibility	20
6.4.4	Reliability	20
6.4.5	Compatibility	20
7	2. Technical Implementation	20
7.1	Contents	20
7.2	Solution Architecture	20
7.2.1	High-Level Architecture	20
7.2.2	System Components	21
7.2.3	Data Flow	21
7.3	Key Technical Decisions	21
7.4	Security Overview	22
8	Technology Stack	22
8.1	Stack Overview	22
8.2	Key Technology Decisions	23
8.2.1	Decision 1: Laravel API	23
8.2.2	Decision 2: React/Vite Telegram Mini App	24
8.2.3	Decision 3: PostgreSQL with PostGIS and pgvector	24
8.3	Development Tools	24
8.4	External Services & APIs	24
9	Deployment & DevOps	25
9.1	Infrastructure	25
9.1.1	Deployment Architecture	25
9.1.2	Environments	25
9.2	CI/CD Pipeline	26
9.2.1	Pipeline Overview	26
9.2.2	Pipeline Steps	26
9.2.3	Example Pipeline Configuration	26
9.3	Environment Variables	27
9.4	How to Run Locally	27
9.4.1	Prerequisites	27
9.4.2	Backend Setup	28
9.4.3	Frontend Setup	28
9.4.4	Database Setup	28
9.5	Monitoring & Logging	28
10	Criteria Documentation Index	29
11	Criterion: Frontend Application	29
11.1	Architecture Decision Record	29
11.1.1	Status	29
11.1.2	Context	29
11.1.3	Decision	29
11.1.4	Alternatives Considered	29
11.1.5	Consequences	30
11.2	Implementation Details	30
11.2.1	Project Structure	30
11.2.2	Key Implementation Decisions	30
11.2.3	Diagrams	30
11.3	Requirements Checklist	30
11.4	Known Limitations	32

11.5 References	32
12 Criterion: Backend API	32
12.1 Architecture Decision Record	32
12.1.1 Status	32
12.1.2 Context	32
12.1.3 Decision	33
12.1.4 Alternatives Considered	33
12.1.5 Consequences	33
12.2 Implementation Details	33
12.2.1 Project Structure	33
12.2.2 Key Implementation Decisions	33
12.2.3 Diagrams	34
12.3 Requirements Checklist	34
12.4 Known Limitations	34
12.5 References	35
13 Criterion: Database Design	35
13.1 Architecture Decision Record	35
13.1.1 Status	35
13.1.2 Context	35
13.1.3 Decision	35
13.1.4 Alternatives Considered	35
13.1.5 Consequences	36
13.2 Implementation Details	36
13.2.1 Main Tables	36
13.2.2 Key Implementation Decisions	36
13.2.3 Diagrams	37
13.3 Requirements Checklist	37
13.4 Known Limitations	37
13.5 References	37
14 Criterion: Real-Time Communication	38
14.1 Architecture Decision Record	38
14.1.1 Status	38
14.1.2 Context	38
14.1.3 Decision	38
14.1.4 Alternatives Considered	38
14.1.5 Consequences	38
14.2 Implementation Details	38
14.2.1 Project Structure	38
14.2.2 Key Implementation Decisions	39
14.2.3 Diagrams	39
14.3 Requirements Checklist	39
14.4 Known Limitations	39
14.5 References	40
15 Criterion: Authentication and Telegram Integration	40
15.1 Architecture Decision Record	40
15.1.1 Status	40
15.1.2 Context	40
15.1.3 Decision	40
15.1.4 Alternatives Considered	40
15.1.5 Consequences	41
15.2 Implementation Details	41

15.2.1 Project Structure	41
15.2.2 Key Implementation Decisions	41
15.2.3 Diagrams	41
15.3 Requirements Checklist	41
15.4 Known Limitations	42
15.5 References	42
16 Criterion: Media Storage and Moderation	42
16.1 Architecture Decision Record	42
16.1.1 Status	42
16.1.2 Context	42
16.1.3 Decision	42
16.1.4 Alternatives Considered	42
16.1.5 Consequences	43
16.2 Implementation Details	43
16.2.1 Project Structure	43
16.2.2 Key Implementation Decisions	44
16.2.3 Diagrams	44
16.3 Requirements Checklist	44
16.4 Known Limitations	45
16.5 References	45
17 Criterion: Containerization and Deployment	45
17.1 Architecture Decision Record	45
17.1.1 Status	45
17.1.2 Context	45
17.1.3 Decision	45
17.1.4 Alternatives Considered	45
17.1.5 Consequences	46
17.2 Implementation Details	46
17.2.1 Project Structure	46
17.2.2 Key Implementation Decisions	46
17.2.3 Diagrams	47
17.3 Requirements Checklist	47
17.4 Known Limitations	47
17.5 References	47
18 3. User Guide	48
18.1 Contents	48
18.2 Getting Started	48
18.2.1 System Requirements	48
18.2.2 Accessing the Application	48
18.2.3 First Launch	48
18.3 Quick Start Guide	49
18.4 User Roles	49
19 Feature Walkthrough	49
19.1 Feature 1: Onboarding and Profile Creation	49
19.1.1 Overview	49
19.1.2 How to Use	49
19.1.3 Tips	50
19.2 Feature 2: Feed and Swiping	50
19.2.1 Overview	50
19.2.2 How to Use	50
19.3 Feature 3: Likes and Matches	50

19.3.1 Overview	50
19.3.2 How to Use	50
19.4 Feature 4: Chat	50
19.4.1 Overview	50
19.4.2 How to Use	51
19.5 Feature 5: Profile Editing and Preferences	51
19.5.1 Overview	51
19.5.2 How to Use	51
19.6 Feature 6: Premium and Payments	51
19.6.1 Overview	51
19.6.2 How to Use	51
19.7 Feature Comparison	51
20 FAQ & Troubleshooting	52
20.1 Frequently Asked Questions	52
20.1.1 General	52
20.1.2 Account & Access	52
20.1.3 Features	53
20.2 Troubleshooting	53
20.2.1 Common Issues	53
20.2.2 Error Messages	53
20.2.3 Browser-Specific Issues	54
20.3 Getting Help	54
20.3.1 Self-Service Resources	54
20.3.2 Contact Support	54
20.3.3 Reporting Bugs	54
21 4. Retrospective	54
21.1 What Went Well	55
21.1.1 Technical Successes	55
21.1.2 Process Successes	55
21.1.3 Personal Achievements	55
21.2 What Did Not Go As Planned	55
21.2.1 Challenges Encountered	56
21.3 Technical Debt & Known Issues	56
21.3.1 Code Quality Issues	56
21.4 Future Improvements	56
21.4.1 High Priority	56
21.4.2 Medium Priority	57
21.4.3 Nice to Have	57
21.5 Lessons Learned	57
21.5.1 Technical Lessons	57
21.5.2 Process Lessons	57
21.5.3 What Would Be Done Differently	58
21.6 Personal Growth	58
21.6.1 Skills Developed	58
21.6.2 Key Takeaways	58
22 5. Appendices	58
22.1 Contents	59
23 Glossary	59
23.1 Acronyms	59
23.2 Domain-Specific Terms	59
23.2.1 Dating Domain	59

23.2.2 Technical Domain	60
24 API Reference	60
24.1 Overview	60
24.2 Public Endpoints	60
24.3 Authenticated Endpoint Groups	60
24.3.1 Current User	61
24.3.2 Moderation	61
24.3.3 Dictionary	61
24.3.4 User Profile	61
24.3.5 Feed	61
24.3.6 Chat	62
24.3.7 Storage	62
24.3.8 Payment	62
24.3.9 Admin	62
24.4 Common Error Responses	62
24.5 OpenAPI Status	63
25 Database Schema	63
25.1 Overview	63
25.2 Entity Relationship Diagram	63
25.3 Tables	63
25.3.1 users	63
25.3.2 user_feed_profile	63
25.3.3 user_search_preferences	64
25.3.4 user_files	64
25.3.5 user_moderation	64
25.3.6 user_swipes	64
25.3.7 chats, chat_users, chat_messages	65
25.3.8 dictionaries	65
25.3.9 subscriptions and transactions	65
25.3.10account lifecycle tables	65
25.4 Relationships	65
25.5 Migrations	66
25.6 Seeding	66

1 PlusDateApp

1.1 Project Information

Field	Value
Student	Mikita Malyshau
Group	22-LR-JS
Supervisor	Andrii Kostromytskyi
Date	May 3, 2026

1.2 Links

Resource	URL
Production	@stage_plusdate_bot

Resource	URL
Repository	PlusDateApp monorepository
API Docs	See API Reference
Design	Plus Date Figma

1.3 Elevator Pitch

PlusDateApp is a Telegram Mini App dating platform for people who want a profile-based discovery experience without leaving Telegram. It solves the problem of fragmented onboarding, profile moderation, swiping, matching, chat, and premium access by combining these flows into one mobile-first web application. The project includes a Laravel API, a React/Vite single page application, PostgreSQL with PostGIS and pgvector support, real-time chat events, Telegram login and bot notifications, media moderation, and subscription flows.

1.4 Evaluation Criteria Checklist

#	Criterion	Status	Documentation
1	Frontend Application	Completed	frontend.md
2	Backend API	Completed	backend-api.md
3	Database Design	Completed	database.md
4	Real-Time Communication	Completed	realtime.md
5	Authentication and Telegram Integration	Completed	telegram-auth.md
6	Media Storage and Moderation	Completed	media-moderation.md
7	Containerization and Deployment	Completed	containerization-deployment.md

1.5 Documentation Navigation

- [Project Overview](#) - business context, goals, users, scope, and requirements.
- [Technical Implementation](#) - architecture, stack, selected criteria, and deployment.
- [User Guide](#) - onboarding and main user scenarios.
- [Retrospective](#) - results, technical debt, lessons learned, and next steps.
- [Appendices](#) - glossary, API reference, and database schema.

Document created: May 3, 2026 Last updated: May 3, 2026

2 1. Project Overview

This section describes the business purpose, users, scope, and functional requirements of PlusDateApp.

2.1 Contents

- [Problem Statement & Goals](#)
- [Stakeholders & Users](#)
- [Scope](#)
- [Features](#)

2.2 Executive Summary

PlusDateApp is a dating product implemented as a Telegram Mini App. The application helps users create a profile, pass moderation, discover compatible profiles, swipe, match, chat, and optionally purchase premium access. The product is designed for mobile-first usage inside Telegram, which reduces registration friction and allows the system to use Telegram identity, bot notifications, and Telegram invoices. The result is a full-stack diploma project with a real backend domain model, a production-style frontend structure, database extensions for location and vector features, and containerized services for local or server deployment.

2.3 Key Highlights

Aspect	Description
Problem	Dating products need a smooth profile, discovery, match, and communication flow, but building it safely requires authentication, moderation, media storage, search, payments, and real-time messaging.
Solution	A Telegram Mini App that combines onboarding, profile editing, swipe feed, matching, chat, premium subscriptions, and moderation in one application.
Target Users	Telegram users looking for dating, administrators/moderators, and project maintainers.
Key Features	Telegram login, onboarding, profile media upload, moderation, swipe feed, likes, matches, chat, premium, account deletion and restore.
Tech Stack	React 19, TypeScript, Vite, Laravel 12, PHP 8.2, PostgreSQL/PostGIS/pgvector, Redis, Reverb, MinIO, Docker.

3 Problem Statement & Goals

3.1 Context

PlusDateApp operates in the dating and social discovery domain. A dating product has to solve several connected problems at once: user identity, profile quality, discovery, preference filtering, matching, communication, moderation, media storage, and monetization. Users expect a simple mobile experience, but the system behind it must enforce safety rules, keep profile data consistent, deliver chat updates quickly, and avoid showing unsuitable profiles.

The project uses Telegram as the entry point. This decision fits the target context because many users already have Telegram installed and can open a Mini App without installing a separate native application. The backend keeps the business logic in a Laravel API, while the frontend focuses on Telegram-specific UX and mobile navigation.

3.2 Problem Statement

Who: Telegram users who want to meet people through a lightweight mobile dating experience.

What: They need profile creation, search preferences, safe media validation, swipe discovery, matches, chat, and premium functions in one reliable flow.

Why: If these functions are separated or poorly integrated, users drop out during onboarding, receive irrelevant profiles, cannot trust profile media, or lose conversation context after matching.

3.2.1 Pain Points

#	Pain Point	Severity	Current Workaround
1	Registration and profile setup can take too long in standalone apps.	High	Users skip setup, use incomplete profiles, or abandon the app.
2	Discovery quality depends on location, age, interests, activity, media, and moderation status.	High	Users manually inspect many irrelevant profiles.
3	Unsafe or low-quality profile media reduces trust.	High	Platforms rely on delayed manual moderation or weak upload rules.
4	Matches lose value when messaging is slow or not synchronized.	Medium	Users switch to external messengers immediately.
5	Premium functions must be monetized without breaking the mobile flow.	Medium	Users are sent to external checkout pages with extra friction.

3.2.2 Before / After

Aspect	Before	After
User entry	User installs or opens a separate dating app, creates credentials, and repeats profile data.	User opens a Telegram Mini App and authenticates through Telegram login data.
Profile quality	Profiles can be incomplete, unverified, or contain unsuitable media.	Onboarding, verification photo, media upload rules, and moderation checks improve profile quality.
Discovery	Users browse generic lists with limited filtering.	Feed uses search preferences, location ordering, hidden swiped profiles, and profile status filters.
Communication	Matching and messaging may be separated.	Match creation and chat features are part of the same backend domain.
Monetization	Premium access is difficult to test and maintain.	Stripe Cashier and Telegram invoice flows are separated behind subscription services.

3.3 Business Goals

Goal	Description	Success Indicator
Reduce onboarding friction	Allow users to start from Telegram and complete a guided profile setup.	User reaches the feed only after required onboarding fields and moderation checks.

Goal	Description	Success Indicator
Improve discovery relevance	Filter feed profiles using preferences, moderation state, previous swipes, and location data.	Feed returns profiles that match stored preferences and excludes already swiped profiles.
Support safe interactions	Add media validation, moderation state, profile visibility rules, and account deletion.	Invalid media or unresolved moderation blocks normal feed access.
Enable real-time communication	Allow matched users to chat and receive message/read updates.	Message events are broadcast through Laravel Reverb/private channels.
Prepare monetization	Add premium features and subscription flows.	Premium page, subscription endpoints, Stripe Cashier, and Telegram invoice path exist.

3.4 Objectives & Metrics

Objective	Metric	Current Value	Target Value	Timeline
Complete onboarding flow	Required onboarding screens implemented	Implemented	100% of required profile fields covered	Diploma scope
Keep feed relevant	Feed filters supported	Age, city, gender, activity, height, eye color, hobbies, premium, video	At least 8 filter types	Diploma scope
Protect media quality	Moderation states and upload validation	Implemented in backend requests, rules, jobs, and moderation screens	All profile media goes through validation/moderation path	Diploma scope
Deliver chat interactions	Core chat operations	Create chat, list chats, send messages, read states	Real-time send and read events	Diploma scope
Make deployment reproducible	Container assets	Backend, SPA, database Docker assets	Docker-based local deployment for all major services	Diploma scope

3.5 Success Criteria

3.5.1 Must Have

- Profile onboarding stores the required user, preference, and media data.
- Feed returns paginated profiles and respects already-swiped exclusions.
- Users can like, dislike, match, respond to likes, and delete matches.
- Users can create chats, send messages, and mark messages as read.
- Media upload and moderation flows prevent unsafe profiles from normal access.
- Backend and frontend can be built and run from documented commands.

3.5.2 Nice to Have

- Premium filters and limits are visible in the user interface.
- Telegram bot notifications are sent for important events.
- Account deletion and restore flows are available.

- Analytics events can be emitted through Google Tag Manager.

3.6 Non-Goals

The project does not aim to implement a native iOS or Android application, a full manual moderation back office, public social networking posts, group chats, machine-learning recommendations beyond vector-ready database support, or a complete production CI/CD pipeline with managed cloud infrastructure.

4 Stakeholders & Users

4.1 Target Audience

Persona	Description	Key Needs
Dating user	A Telegram user who wants to meet people through a mobile-first interface.	Fast start, clear onboarding, relevant feed, safe media, chat after matching.
Premium user	A highly active user who wants fewer limits and more control over discovery.	More swipes, advanced filters, revert actions, visibility of likes, premium explanations.
Moderator or administrator	A project operator responsible for user safety and test data.	Account management, profile creation for bot/test users, moderation status visibility.
Developer or maintainer	A person maintaining the codebase after the diploma defense.	Clear architecture, reproducible setup, isolated services, documented endpoints and database schema.

4.2 User Personas

4.2.1 Persona 1: Anna, Regular User

Attribute	Details
Role	End user looking for matches
Age	20-30
Tech Savviness	Medium
Goals	Create a profile quickly, see people nearby, avoid unsafe profiles, and chat after a match.
Frustrations	Long registration, unclear profile requirements, irrelevant profiles, slow messaging.
Scenario	Anna opens the Mini App from Telegram, completes onboarding, uploads photos and a verification photo, waits for moderation, then uses the feed and chat.

4.2.2 Persona 2: Mark, Premium-Oriented User

Attribute	Details
Role	Active dating app user

Attribute	Details
Age	25-38
Tech Savviness	High
Goals	Tune search preferences, see likes, revert an accidental dislike, and use premium filters.
Frustrations	Hard limits, weak filters, and payment flows that interrupt mobile usage.
Scenario	Mark updates height, activity, city, eye color, and interest preferences, opens the premium page, and buys access through a supported subscription flow.

4.2.3 Persona 3: Project Maintainer

Attribute	Details
Role	Developer or technical reviewer
Age	Any
Tech Savviness	High
Goals	Understand the architecture, run the services locally, verify API behavior, and inspect data relationships.
Frustrations	Hidden dependencies, undocumented routes, missing environment variables, and unclear service boundaries.
Scenario	The maintainer starts PostgreSQL, Redis, MinIO, the Laravel API, and the Vite SPA, then tests login, feed, upload, and chat flows.

4.3 Stakeholder Map

4.3.1 High Influence / High Interest

- Student developer: responsible for implementation, documentation, and defense.
- Supervisor and assessment committee: evaluate the business scope, technical quality, documentation, and honesty of results.

4.3.2 High Influence / Low Interest

- External service providers: Telegram, Stripe, storage providers, Sentry, and hosting platforms affect availability and constraints but are not involved in daily product decisions.

4.3.3 Low Influence / High Interest

- End users: they directly benefit from the product and provide feedback, but they do not decide architecture.
- Future maintainers: they rely on code quality and documentation after the defense.

4.3.4 Low Influence / Low Interest

- Casual visitors: they may see the application through a shared link but are not part of the primary product flow until they authenticate and onboard.

5 Project Scope

5.1 In Scope

Feature	Description	Priority
Telegram authentication	Login endpoint and frontend initialization for Telegram Mini App context.	Must
Onboarding	Guided steps for basic data, age, city, interests, media, and verification.	Must
Profile management	View and edit profile fields, photos, video, search preferences, settings, referral, deletion, and restore flow.	Must
Feed and swipes	Profile discovery, like, dislike, superlike-oriented UI, revert swipe, match handling, and available swipe counters.	Must
Likes and matches	List incoming likes, respond to likes, and remove matches.	Must
Chat	Chat list, message list, send messages, unread counts, and read state updates.	Must
Media storage	Photo/video upload, main photo selection, delete file, MinIO/external storage abstraction, and blurred-photo job.	Should
Moderation	User moderation status, photo validation, moderation screens, and Telegram moderation webhook.	Should
Subscriptions	Premium page, current subscription, subscribe, cancel, Stripe Cashier, Telegram invoice path.	Should
Containerized setup	Docker assets for backend, frontend, database, Redis, and MinIO.	Should

5.2 Out of Scope

Feature	Reason	When Possible
Native mobile apps	The chosen delivery channel is Telegram Mini App plus responsive SPA.	Future phase
Full moderation dashboard	Admin endpoints exist, but a complete web dashboard is beyond diploma scope.	Future phase
Multi-language documentation	Thesis must be in English; app localizations exist for English and Russian UI texts.	Not required for thesis

Feature	Reason	When Possible
Advanced ML recommendation engine	Database is prepared for vectors, but a full training/recommendation system is too large for this scope.	Future phase
Group chats and social feed	The product focuses on one-to-one dating matches.	Future phase

5.3 Assumptions

#	Assumption	Impact if Wrong	Probability
1	Users are comfortable opening the product through Telegram.	A standalone login and distribution channel would be required.	Medium
2	PostgreSQL with PostGIS and pgvector is acceptable for the data layer.	Location and vector-ready features would need redesign.	Low
3	External services such as Telegram, Stripe, and object storage are available in the target environment.	Login, invoices, notifications, or uploads may fail without mocks.	Medium
4	Mobile-first UX is more important than desktop-specific screens.	Desktop layout would require additional design work.	Low

5.4 Constraints

Constraint Type	Description	Mitigation
Time	Diploma project has a fixed build and assessment period.	Scope focuses on a complete vertical slice instead of native apps and large admin panels. Environment variables and local Docker services are used where possible. Frontend initialization handles viewport, theme, safe area, closing behavior, and back button behavior. Code is organized into clear backend services and feature-sliced frontend modules.
Budget	Paid external services should be optional or configurable.	
Technology	Telegram Mini App imposes webview and SDK constraints.	
Resources	Project is implemented as a student project.	

Constraint Type	Description	Mitigation
External	Telegram, Stripe, storage, and Sentry depend on credentials and network access.	Integrations are isolated behind controllers, services, configuration, and environment variables.

5.5 Dependencies

Dependency	Type	Owner	Status
Telegram Bot API and Mini App environment	External	Telegram	Implemented
Stripe / Laravel Cashier	External	Stripe and Laravel ecosystem	Implemented
PostgreSQL/PostGIS/pgvector	Technical	PostgreSQL ecosystem	Implemented
Redis	Technical	Redis ecosystem	Implemented
MinIO or S3-compatible storage	Technical	Storage provider	Implemented
Laravel Reverb	Technical	Laravel ecosystem	Implemented
React/Vite/TanStack Query/Zustand	Technical	JavaScript ecosystem	Implemented

6 Features & Requirements

6.1 Epics Overview

Epic	Description	Stories	Status
E1: Identity and onboarding	Telegram login, user creation, required profile setup, and moderation entry.	5	Completed
E2: Profile and preferences	Profile view/edit, media, interests, activity, city, height, age, eye color, and search settings.	7	Completed
E3: Discovery and matching	Feed profiles, swipe actions, likes, matches, revert action, and limits.	6	Completed
E4: Communication	Chat list, chat detail, message sending, unread states, and real-time events.	4	Completed
E5: Safety and monetization	Moderation, storage validation, premium page, subscriptions, and payment callbacks.	6	Completed

6.2 User Stories

6.2.1 Epic 1: Identity and Onboarding

This epic ensures a user can enter the application through Telegram and complete the minimum profile data required before using the feed.

ID	User Story	Acceptance Criteria	Priority	Status
US-001	As a Telegram user, I want to log in through Telegram so that I do not create a separate password.	Login endpoint exists; frontend initializes Telegram context; authenticated API calls use Sanctum.	Must	Completed
US-002	As a new user, I want a guided onboarding flow so that I know what data is required.	Routes exist for basic, age, city, interests, media, and verification steps.	Must	Completed
US-003	As a user, I want to upload profile media so that my profile is attractive and trustworthy.	Photo/video upload endpoints and frontend inputs exist; upload rules validate files.	Must	Completed
US-004	As the system, I want to block incomplete or unresolved users from the feed so that the app remains safe.	Router guard redirects non-onboarded users and unresolved moderation cases.	Must	Completed
US-005	As a returning deleted user, I want to restore my account state when possible.	Restore screen exists and backend stores deletion snapshots.	Should	Partially completed

6.2.2 Epic 2: Profile and Preferences

This epic lets users describe themselves and control which profiles they want to see.

ID	User Story	Acceptance Criteria	Priority	Status
US-006	As a user, I want to edit my profile fields so that my data stays current.	Profile edit routes and backend profile upsert endpoint exist.	Must	Completed
US-007	As a user, I want to set city, activity, interests, height, age, and eye color so that matching is more relevant.	Dedicated screens and search preference endpoints exist.	Must	Completed
US-008	As a user, I want to choose a main profile photo so that others see the best media first.	Backend endpoint for main photo selection and media UI exist.	Should	Completed

ID	User Story	Acceptance Criteria	Priority	Status
US-009	As a user, I want to delete my account so that I control my data.	Delete account screen and backend endpoint exist.	Should	Completed

6.2.3 Epic 3: Discovery and Matching

The feed is the main dating workflow. It must show relevant profiles and handle user decisions.

ID	User Story	Acceptance Criteria	Priority	Status
US-010	As a user, I want to browse profile cards so that I can discover people.	Feed page, swipe card stack, and feed profiles endpoint exist.	Must	Completed
US-011	As a user, I want to like or dislike a profile so that the system learns my decision.	Swipe endpoint stores action and returns match result when applicable.	Must	Completed
US-012	As a user, I want matched users to become available for chat.	Match and chat services exist; match events are broadcast.	Must	Completed
US-013	As a premium user, I want to revert an accidental dislike.	Revert dislike UI and backend revert endpoint exist.	Should	Completed
US-014	As a user, I want to see incoming likes.	Likes page and respond-to-like endpoint exist.	Should	Completed

6.2.4 Epic 4: Communication

Communication completes the dating loop after a match.

ID	User Story	Acceptance Criteria	Priority	Status
US-015	As a matched user, I want to open a chat list.	Chats page and user chats endpoint exist.	Must	Completed
US-016	As a matched user, I want to send messages.	Message form, send message endpoint, and message event exist.	Must	Completed
US-017	As a matched user, I want unread states to update.	Unread count query, mark-read endpoint, and read event exist.	Should	Completed
US-018	As a user outside the app, I want Telegram notifications for important chat events.	Notification job and Telegram service exist.	Should	Completed

6.2.5 Epic 5: Safety and Monetization

This epic covers operations that make the product more realistic as a business application.

ID	User Story	Acceptance Criteria	Priority	Status
US-019	As the platform, I want to moderate uploaded photos.	Moderation jobs, validation rules, moderation status, and moderation UI exist.	Must	Completed
US-020	As a user, I want clear feedback when moderation blocks access.	Router guard and moderation screens exist.	Must	Completed
US-021	As a premium user, I want to subscribe.	Premium page, current subscription, subscribe, cancel, Stripe Cashier, and Telegram invoice endpoint exist.	Should	Completed
US-022	As an administrator, I want to create bot/test profiles.	Admin create-bot endpoint exists.	Could	Completed

6.3 Use Case Diagram

6.4 Non-Functional Requirements

6.4.1 Performance

Requirement	Target	Measurement Method
Initial SPA build	Vite production build succeeds without TypeScript errors.	npm run build in spa.
API response time	Core endpoints should respond within acceptable interactive latency on local Docker setup.	Manual API testing and Laravel logs.
Feed pagination	Feed should use cursor pagination and not load all profiles at once.	Backend service uses cursorPaginate.
Chat pagination	Message history should load in pages.	Backend service uses cursor pagination for chats and messages.

6.4.2 Security

- Telegram-based login data is handled by the backend authentication controller.
- API routes are protected with `auth:sanctum` where user identity is required.
- Request classes and rules validate profile, file, feed, chat, and account actions.
- File deletion and profile visibility use rule classes instead of trusting the client.
- Secrets are expected in `.env` files and must not be committed.

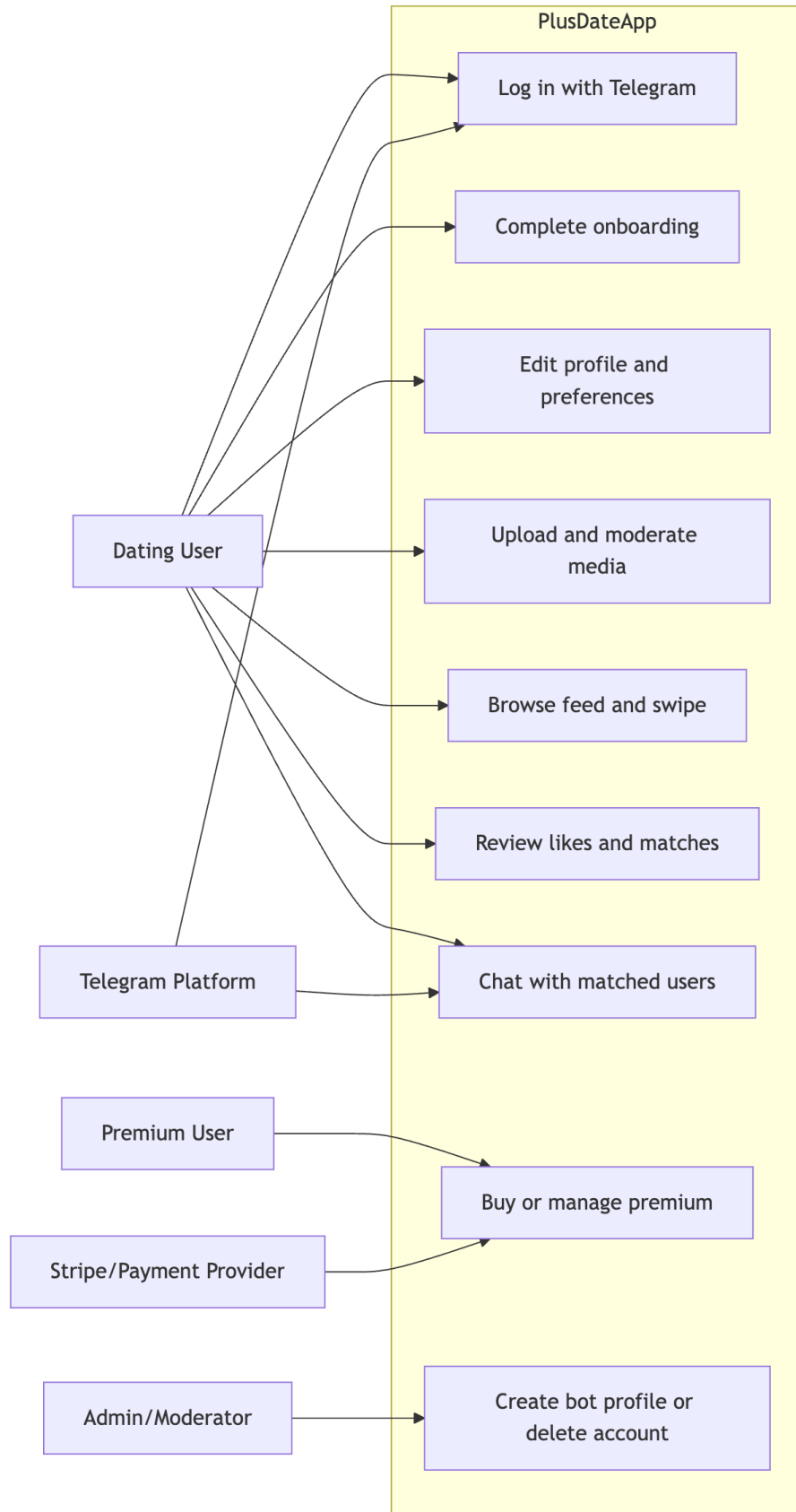


Figure 1: Diagram

6.4.3 Accessibility

- The application is mobile-first and uses large touch targets for swipe, footer navigation, forms, and media actions.
- Form validation errors are exposed through shared input and error components.
- Further accessibility audit with real devices and screen readers is listed as future work.

6.4.4 Reliability

Metric	Target
Service restart	Containers can be rebuilt from Dockerfiles and compose files.
Data persistence	PostgreSQL, Redis, and MinIO use Docker volumes in local setup.
Background work	Queues process moderation, notifications, and file-related jobs.
Error visibility	Sentry package is included for backend error tracking configuration.

6.4.5 Compatibility

Platform/Browser	Minimum Version
Telegram mobile webview	Current supported Telegram app versions
Chrome	Current stable
Safari	Current stable on iOS
Android WebView	Current supported Android WebView

7 2. Technical Implementation

This section describes the architecture, technology choices, implementation decisions, selected assessment criteria, and deployment approach for PlusDateApp.

7.1 Contents

- [Tech Stack](#)
- [Criteria Documentation](#)
- [Deployment](#)

7.2 Solution Architecture

7.2.1 High-Level Architecture

The frontend is a single page application designed for Telegram's mobile webview. It communicates with the Laravel API over HTTP and subscribes to private real-time channels through Laravel Echo/Reverb. The backend owns domain logic for profiles, feed, swipes, chats, moderation, files, subscriptions, dictionaries, and administration. PostgreSQL stores structured data, PostGIS supports distance-based city filtering, pgvector prepares the profile table for vector features, Redis supports queues/cache, and MinIO or S3-compatible storage stores uploaded media.

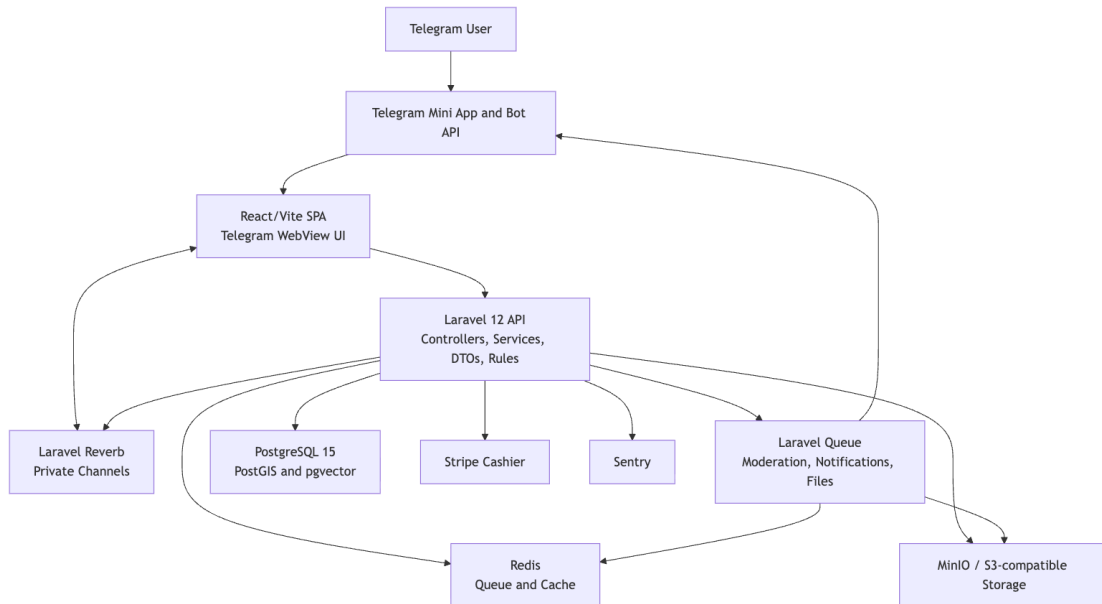
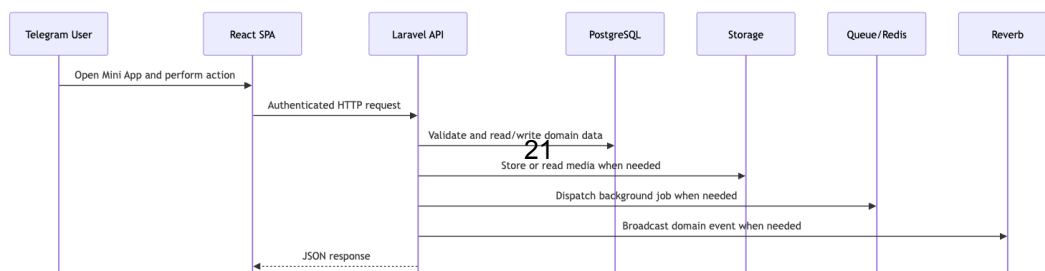


Figure 2: Diagram

7.2.2 System Components

Component	Description	Technology
Frontend SPA	Mobile-first Telegram Mini App UI with onboarding, profile, feed, likes, chat, premium, moderation, and settings screens.	React 19, TypeScript, Vite 7, React Router, TanStack Query, Zustand, i18next, Tailwind CSS 4
Backend API	REST API and business workflows for authentication, users, feed, chat, storage, moderation, payments, dictionaries, admin, and bot webhooks.	Laravel 12, PHP 8.2, Sanctum, DTOs, services, rules, Eloquent
Database	Persistent relational data with geospatial and vector extensions.	PostgreSQL 15, PostGIS, pgvector
Realtime	Private event channels for chat, matches, moderation, and user updates.	Laravel Reverb, Laravel Echo
Queue and Cache	Background jobs for moderation, media processing, notifications, and system tasks.	Redis, Laravel Queue
Storage	User photo/video storage and derived blurred images.	MinIO locally, S3/GCS-compatible abstractions
External Services	Identity, notifications, invoices, payments, error tracking.	Telegram Bot API, Stripe Cashier, Sentry

7.2.3 Data Flow



Decision	Rationale	Alternatives Considered
Use Laravel as backend	Strong API, queues, events, validation, Eloquent ORM, Sanctum, Cashier, and Reverb support.	Node.js/NestJS, Django, Spring Boot
Use React/Vite SPA	Fast iteration, TypeScript support, rich mobile UI, feature-sliced frontend structure.	Vue, Next.js, server-rendered Blade
Use PostgreSQL with PostGIS and pgvector	One database supports relational data, distance ordering, and vector-ready profile features.	MySQL, MongoDB, separate vector DB
Use service and DTO layers	Keeps controllers thin and makes business flows explicit.	Fat controllers, direct model manipulation from frontend-specific controllers
Use Docker assets	Reproducible local setup for API, database, Redis, MinIO, and SPA.	Manual local installation only

7.4 Security Overview

Aspect	Implementation
Authentication	Telegram login endpoint and Laravel Sanctum protected routes.
Authorization	Route middleware, request validation, and domain rule classes such as file, profile, feed, and storage rules.
Data Protection	HTTPS is expected in production; secrets are environment variables; uploaded files are handled by storage services.
Input Validation	Laravel form requests, enum-based validation, zod schemas in frontend forms, and custom rule classes.
Secrets Management	.env files for local development; deployment should use platform secrets.
Moderation	User moderation records, moderation jobs, and frontend route guards block unresolved high-risk states.

8 Technology Stack

8.1 Stack Overview

Layer	Technology	Version	Justification
Frontend	React	19.1	Component model fits a multi-screen mobile SPA.
Frontend language	TypeScript	5.8	Strong typing for API models, forms, and shared UI contracts.
Build tool	Vite	7.0	Fast local development and simple production build.

Layer	Technology	Version	Justification
Routing	React Router	7.7	Nested route tree supports onboarding, moderation, profile edit, feed, chat, and premium screens.
Server state	TanStack Query	5.83	Caching, loading states, and API query invalidation for SPA data.
Client state	Zustand	5.0	Lightweight local state for flows such as onboarding, moderation, and swipe cards.
UI styling	Tailwind CSS	4.1	Utility-first styling for mobile layouts and shared UI components.
Backend language	PHP	8.2+	Required by Laravel 12 and supported by common hosting/container stacks.
Backend framework	Laravel	12.0	Controllers, requests, validation, events, queues, ORM, Sanctum, Cashier, and Reverb ecosystem.
Authentication	Laravel Sanctum plus Telegram login	Sanctum 4.0	Token/session API protection with Telegram identity as the entry mechanism.
Database	PostgreSQL/PostGIS	15	Relational data plus geospatial location support.
Vector extension	pgvector	Database extension	Prepares profile data for vector similarity and feed features.
ORM	Eloquent	Laravel 12	First-class Laravel model and migration support.
Cache/Queue	Redis	7-alpine	Queue backend and cache layer for background work.
Realtime	Laravel Reverb / Echo	Reverb 1.0	Private WebSocket-style events for chat, matches, moderation, and user updates.
Storage	MinIO / S3-compatible storage	Latest locally	Local object storage with deployable S3/GCS-compatible abstraction.
Payments	Laravel Cashier / Stripe	Cashier 15.7	Subscription lifecycle and payment integration.
Deployment	Docker, Nginx, Supervisor/Octane config	Project files	Reproducible services and production-style runtime layout.

8.2 Key Technology Decisions

8.2.1 Decision 1: Laravel API

Context: The backend needed authenticated APIs, validation, database migrations, queues, events, file handling, payments, and real-time broadcasting.

Decision: Use Laravel 12 with service classes, DTOs, Eloquent models, form requests, custom rules, queues, events, Sanctum, Reverb, and Cashier.

Rationale: - Laravel gives a complete backend toolkit without building infrastructure primitives manually. - Form requests and rule classes make validation explicit and reusable. - Eloquent migrations map well to a relational dating-domain model. - Events and queues fit chat, moderation, notification, and file workflows.

Trade-offs: - Pros: productive full-stack backend, mature ecosystem, strong conventions. - Cons: requires PHP runtime knowledge and careful service boundaries to avoid oversized controllers.

8.2.2 Decision 2: React/Vite Telegram Mini App

Context: The application needed a mobile-first, highly interactive UI with Telegram-specific initialization, swiping, modals, animations, media upload, and chat.

Decision: Use React 19, TypeScript, Vite, React Router, TanStack Query, Zustand, i18next, Framer Motion, and Telegram Mini App SDK.

Rationale: - React handles the large number of screens and shared UI components. - TypeScript reduces integration mistakes with API resources. - Vite keeps development fast. - Feature-sliced folders make the frontend understandable as the number of flows grows.

8.2.3 Decision 3: PostgreSQL with PostGIS and pgvector

Context: Dating discovery needs structured profile data, preferences, location-aware ordering, and future recommendation support.

Decision: Use PostgreSQL as the primary database and enable PostGIS and pgvector.

Rationale: - Relational tables are appropriate for users, profiles, files, swipes, chats, subscriptions, and moderation records. - PostGIS allows distance calculations from city coordinates. - pgvector keeps vector-ready data close to the profile table instead of adding a separate vector service too early.

8.3 Development Tools

Tool	Purpose	Notes
Git	Version control	Repository prepared as a step-by-step diploma history.
npm	Frontend dependencies and scripts	npm install, npm run dev, npm run build, npm run lint.
Composer	Backend dependencies and scripts	composer install, composer test, Laravel package discovery.
ESLint/Prettier	Frontend code quality	Configured in SPA package.
PHPUnit	Backend tests	Feature tests exist for feed and user profile controllers.
Docker Compose	Local service orchestration	Backend stack includes API, PostgreSQL, Redis, and MinIO.
Laravel Artisan	Backend development	Migrations, route list, queues, tests, cache/config commands.
MkDocs	Documentation structure	Template contains mkdocs.yml for documentation site/PDF workflows.

8.4 External Services & APIs

Service	Purpose	Pricing Model
Telegram Bot API / Mini App	Login context, bot webhooks, notifications, and invoice flow.	External platform; project uses credentials.
Stripe	Subscription payments through Laravel Cashier.	Paid/payment provider model.
Sentry	Backend error tracking integration.	Free tier and paid tiers available.
MinIO/S3/GCS-compatible storage	User media storage.	Local MinIO for development; provider-dependent in production.
Google Tag Manager	Frontend analytics event tracking.	External analytics service.

9 Deployment & DevOps

9.1 Infrastructure

9.1.1 Deployment Architecture

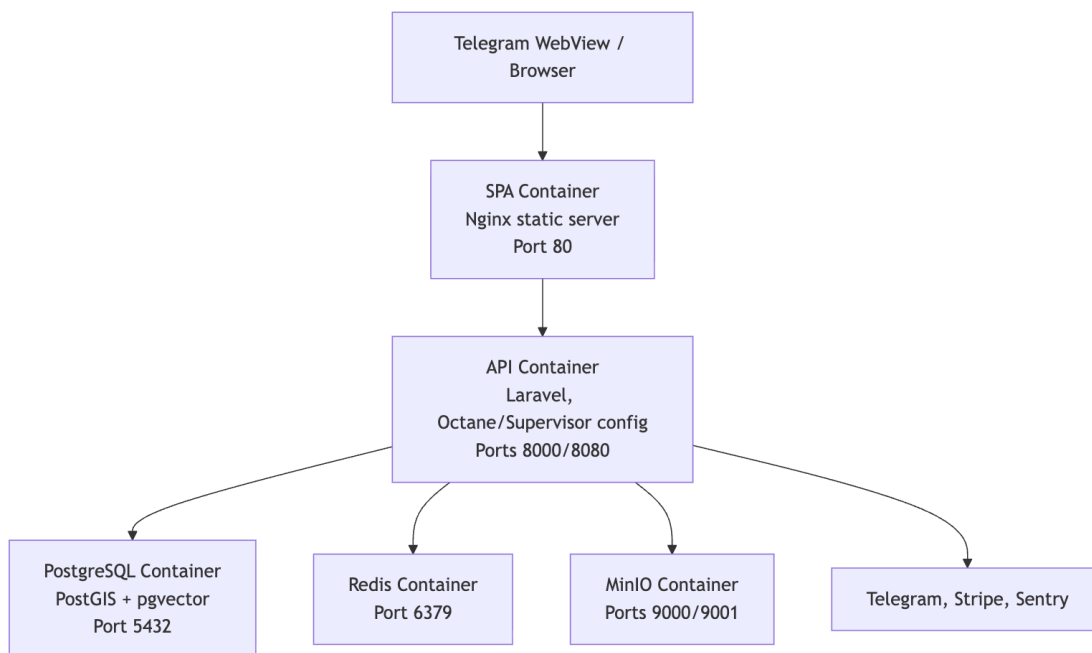


Figure 4: Diagram

The repository contains separate Docker assets for the API, SPA, and database bootstrap. The backend compose stack starts Laravel, PostgreSQL/PostGIS, Redis, and MinIO. The SPA compose stack builds the Vite application and serves the generated static files with Nginx.

9.1.2 Environments

Environment	URL	Branch
Development API	http://localhost:8000	feature branch
Development SPA	http://localhost:5173 for Vite or http://localhost for Nginx container	feature branch
Development database	localhost:5432	local Docker volume
Production	Telegram bot @stage_plusdate_bot	main/release branch

9.2 CI/CD Pipeline

9.2.1 Pipeline Overview



Figure 5: Diagram

The repository does not include a finished hosted CI configuration. The documented pipeline below is the expected production pipeline based on the available scripts and Docker files.

9.2.2 Pipeline Steps

Step	Tool	Actions
Install	Composer, npm	Install backend and frontend dependencies.
Lint	ESLint	Run <code>npm run lint</code> in spa.
Test	PHPUnit/Laravel	Run <code>php artisan test</code> or <code>composer test</code> in api.
Build SPA	TypeScript, Vite	Run <code>npm run build</code> in spa.
Build containers	Docker	Build API, SPA, and database images where needed.
Deploy	Target hosting platform	Push images or run compose stack with environment variables.
Verify	HTTP smoke checks	Call <code>/api/healthcheck</code> , open SPA, and test authentication-dependent flows.

9.2.3 Example Pipeline Configuration

```

name: plusdate-ci

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  verify:
    runs-on: ubuntu-latest
  
```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 22
  - name: Install SPA dependencies
    working-directory: spa
    run: npm ci
  - name: Lint SPA
    working-directory: spa
    run: npm run lint
  - name: Build SPA
    working-directory: spa
    run: npm run build
  - name: Build backend image
    working-directory: api
    run: docker compose build

```

9.3 Environment Variables

Variable	Description	Required	Example
APP_NAME	Laravel application name	Yes	PlusDate
APP_URL	Public API URL	Yes	https://api.example.com
DB_CONNECTION	Database driver	Yes	pgsql
DB_HOST	Database host	Yes	postgres
DB_DATABASE	Database name	Yes	plusdate
DB_USERNAME	Database username	Yes	plusdate
DB_PASSWORD	Database password	Yes	Stored as secret
REDIS_HOST	Redis host	Yes	redis
MINIO_ROOT_USER	Local object storage username	Local	Stored as secret
MINIO_ROOT_PASSWORD	Local object storage password	Local	Stored as secret
VITE_API_URL	SPA API base URL	Yes	https://api.example.com/api
VITE_SOCKET_AUTH_URL	WebSocket auth endpoint	Yes	https://api.example.com/broad
VITE_SOCKET_HOST	WebSocket host	Yes	api.example.com
VITE_SOCKET_PORT	WebSocket port	Yes	443
VITE_GTM_TOKEN	Google Tag Manager token	Optional	Stored as secret

Secrets Management: Local development uses `.env` files. Production deployment should use hosting platform secrets or CI/CD secrets. Real Telegram, Stripe, storage, database, and Sentry credentials must not be committed.

9.4 How to Run Locally

9.4.1 Prerequisites

- Node.js 22+
- PHP 8.2+
- Composer
- Docker and Docker Compose
- PostgreSQL client tools are optional but useful for inspection

9.4.2 Backend Setup

```
cd api
cp .env.example .env
composer install
npm install
php artisan key:generate
docker compose up -d --build
php artisan migrate --seed
php artisan queue:listen --tries=1
```

Useful checks:

```
php artisan test
php artisan route:list
curl http://localhost:8000/api/healthcheck
```

9.4.3 Frontend Setup

```
cd spa
cp .env.example .env
npm install
npm run dev
```

For a production-style local frontend container:

```
cd spa
docker compose up -d --build
```

9.4.4 Database Setup

```
cd database
docker compose up -d --build
```

The database bootstrap enables postgis, postgis_topology, and vector extensions.

9.5 Monitoring & Logging

Aspect	Tool	Notes
Application logs	Laravel logs / Laravel Pail	Local development and debugging.
Error tracking	Sentry Laravel package	Dependency is included; production DSN should be configured through environment variables.
Queue visibility	Laravel queue logs	Queue workers process moderation, notification, and file jobs.
Health check	/api/healthcheck	Returns JSON status for smoke tests.

10 Criteria Documentation Index

This folder contains one Architecture Decision Record for each selected diploma evaluation criterion. The original template structure was kept, but the starter file was replaced with this index so the documentation contains only project-specific content.

#	Criterion	ADR
1	Frontend Application	frontend.md
2	Backend API	backend-api.md
3	Database Design	database.md
4	Real-Time Communication	realtime.md
5	Authentication and Telegram Integration	telegram-auth.md
6	Media Storage and Moderation	media-moderation.md
7	Containerization and Deployment	containerization-deployment.md

11 Criterion: Frontend Application

11.1 Architecture Decision Record

11.1.1 Status

Status: Accepted

Date: May 3, 2026

11.1.2 Context

PlusDateApp needs a mobile-first interface that works inside Telegram's webview and supports many stateful flows: onboarding, moderation, feed swiping, profile editing, search preferences, likes, chats, premium screens, account deletion, and restoration. The frontend must feel like a product, not only an API test client, because the dating workflow depends heavily on fast navigation, media previews, touch actions, and clear screen transitions.

11.1.3 Decision

The frontend is implemented as a React 19 and TypeScript SPA built with Vite. It uses React Router for nested screens, TanStack Query for server state, Zustand for local workflow state, i18next for localization, Framer Motion for transitions, Tailwind CSS for styling, Telegram Mini App SDK for Telegram-specific behavior, and a feature-sliced folder structure.

11.1.4 Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Native mobile app	Best device integration	Requires separate iOS/Android builds and app store delivery	Too large for diploma scope and weaker Telegram distribution fit

Alternative	Pros	Cons	Why Not Chosen
Server-rendered Laravel Blade	Simple deployment with backend	Less suitable for swipe-heavy SPA and Telegram webview state	Product requires interactive mobile UI
Next.js	Strong routing and SSR	Extra SSR complexity not needed for Telegram Mini App	Vite SPA is simpler and sufficient

11.1.5 Consequences

Positive: - Clear route tree for all product screens. - Strong typing for UI models and API responses. - Fast local development and production builds. - Frontend code is organized by app, pages, widgets, features, entities, and shared modules.

Negative: - SPA behavior depends on correct API and Telegram initialization. - More client-side state must be managed carefully.

Neutral: - SEO is not a major target because the primary channel is Telegram Mini App usage.

11.2 Implementation Details

11.2.1 Project Structure

```
spa/src/
├─ app/           # bootstrap, providers, routes, styles, config
├─ pages/         # full user screens
├─ processes/     # onboarding and moderation processes
├─ widgets/       # page layout, action menu, camera modal, payment modals
├─ features/      # swipe cards and user interactions
├─ entities/      # user, chats, likes, dictionary, profile components
└─ shared/        # API client, sockets, UI kit, hooks, assets, types
```

11.2.2 Key Implementation Decisions

Decision	Rationale
Feature-sliced structure	Keeps a large SPA navigable and separates domain entities from pages.
RouterGuard	Centralizes onboarding, deleted account, moderation, and footer visibility rules.
TanStack Query	Avoids manual loading/error/cache code for API-heavy screens.
Shared UI kit	Keeps buttons, inputs, selectors, media carousel, tabs, and modals consistent.

11.2.3 Diagrams

11.3 Requirements Checklist

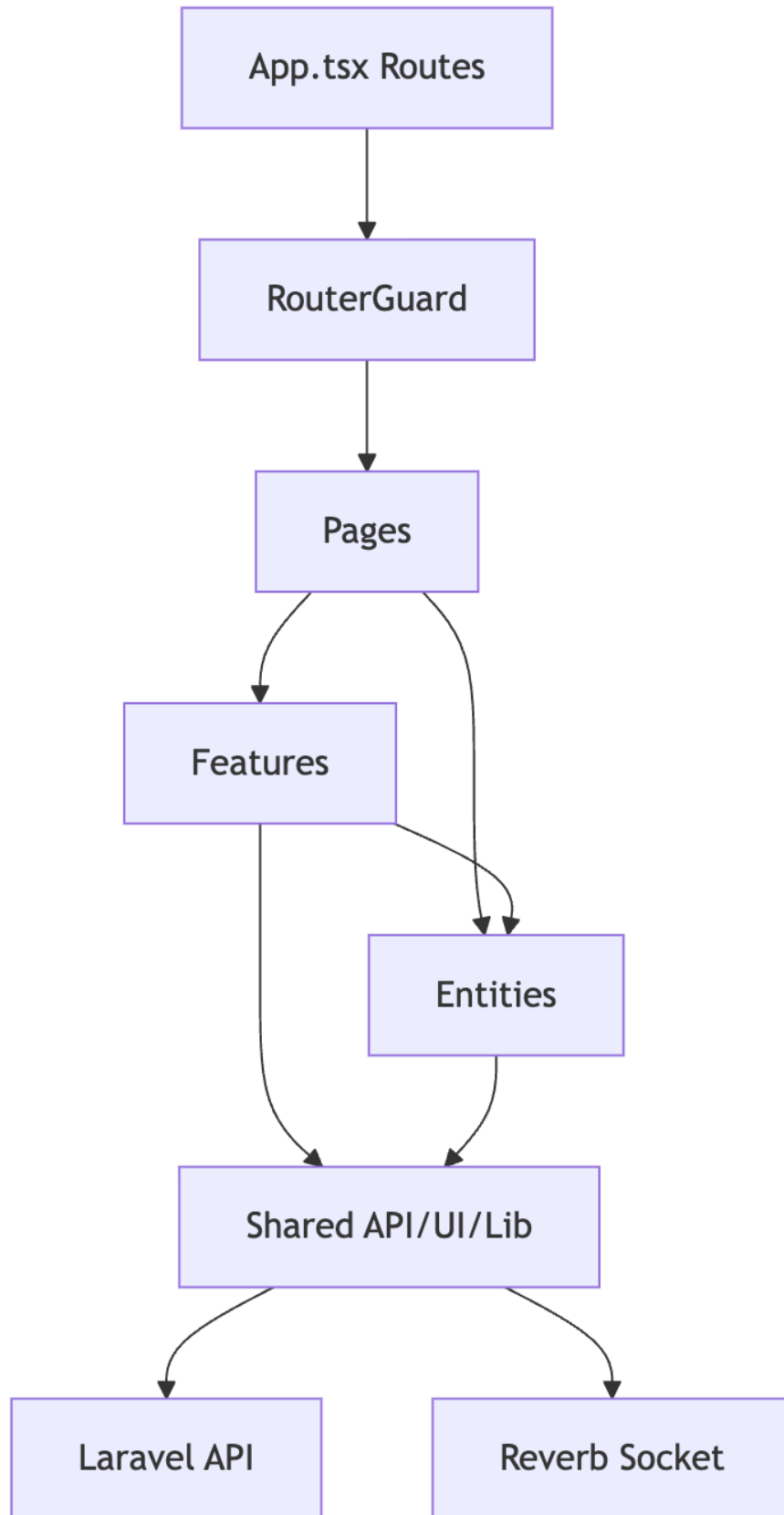


Figure 6: Diagram
31

#	Requirement	Status	Evidence/Notes
1	SPA with multiple screens	Completed	spa/src/app/App.tsx defines onboarding, moderation, feed, profile, likes, premium, chats, and guest routes.
2	Typed frontend	Completed	TypeScript project with entity and API types.
3	API integration	Completed	Shared Axios instance and query modules under entities and features.
4	State management	Completed	TanStack Query and Zustand are used.
5	Mobile/Telegram behavior	Completed	Telegram initialization and back button/safe area behavior exist.
6	Build and lint scripts	Completed	npm run build and npm run lint are defined.

11.4 Known Limitations

Limitation	Impact	Potential Solution
No formal E2E suite in repository	UI regressions require manual checks	Add Playwright tests for onboarding, feed, and chat.
Accessibility audit not documented	Some mobile UI details may need improvement	Run keyboard/screen-reader checks and fix shared components.

11.5 References

- spa/src/app/App.tsx
- spa/src/app/router/ui/RouterGuard.tsx
- spa/src/shared/ui/
- spa/package.json

12 Criterion: Backend API

12.1 Architecture Decision Record

12.1.1 Status

Status: Accepted

Date: May 3, 2026

12.1.2 Context

The product needs a backend that can handle authenticated user data, profile updates, feed logic, swipe decisions, chat, media files, payments, bot webhooks, dictionaries, moderation, and admin functions. This is more than CRUD: many operations require validation, transactions, background jobs, events, and rule-based access checks.

12.1.3 Decision

The backend is implemented with Laravel 12 and PHP 8.2. Routes are grouped by domain in routes/api.php, controllers delegate business logic to services, request objects validate input, DTOs structure data transfer, Eloquent models persist domain state, and custom rule classes enforce product constraints.

12.1.4 Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Node.js/NestJS	TypeScript across stack, modular	More setup for queues/payments/broadcasting/database conventions	Laravel already provides needed backend features
Django REST Framework	Mature ORM and admin	Python stack differs from existing repository direction	Less direct fit for Reverb/Cashier ecosystem
Fat Laravel controllers	Faster initial coding	Harder to test and maintain	Services and DTOs make flows clearer

12.1.5 Consequences

Positive: - Domain routes are easy to inspect. - Validation is close to HTTP boundaries. - Services isolate business workflows such as feed, chat, user profile, moderation, storage, and subscriptions. - Laravel features reduce custom infrastructure code.

Negative: - The service layer must be kept disciplined to avoid duplication. - API documentation is currently manual instead of generated from OpenAPI annotations.

Neutral: - The API is designed for the SPA, not as a public third-party developer platform.

12.2 Implementation Details

12.2.1 Project Structure

```
api/
├── routes/api.php
├── app/Http/Controllers/
├── app/Http/Requests/
├── app/Http/Resources/
├── app/Services/
├── app/Dto/
├── app/Models/
├── app/Rules/
├── app/Events/
└── app/Jobs/
```

12.2.2 Key Implementation Decisions

Decision	Rationale
Domain route groups	Authentication, user, feed, chat, storage, payment, moderation, admin, and dictionary APIs are separated.
Service classes DTO and AutoMapper usage	Business operations are kept out of controllers. Reduces loose arrays and clarifies request/response transformations.
Transactions in services	Chat creation, message sending, and profile upserts protect data consistency.

12.2.3 Diagrams

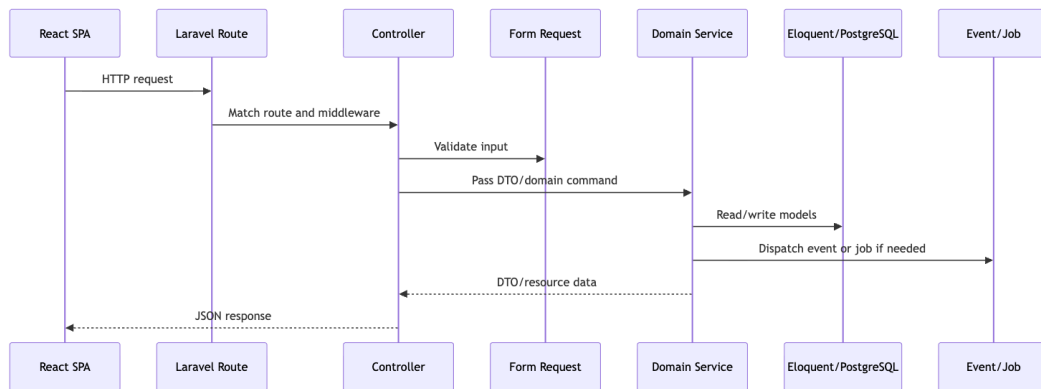


Figure 7: Diagram

12.3 Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	REST API routes	Completed	api/routes/api.php defines all main endpoints.
2	Authentication protection	Completed	Protected group uses auth:sanctum and activity middleware.
3	Validation	Completed	Request classes and rule classes exist by domain.
4	Business services	Completed	Services exist for user, feed, chat, storage, moderation, dictionary, admin, subscriptions.
5	Tests	Partially completed	Feature tests exist for feed and profile controllers; more coverage is needed.
6	Health endpoint	Completed	/api/healthcheck returns JSON status.

12.4 Known Limitations

Limitation	Impact	Potential Solution
OpenAPI generation is not configured	API docs must be maintained manually	Add OpenAPI annotations or generate docs from routes/resources.
Test coverage is incomplete	Some regressions may be missed	Add tests for chat, payments, moderation, and storage.

12.5 References

- `api/routes/api.php`
- `api/app/Http/Controllers/`
- `api/app/Services/`
- `api/app/Dto/`
- `api/tests/Feature/`

13 Criterion: Database Design

13.1 Architecture Decision Record

13.1.1 Status

Status: Accepted

Date: May 3, 2026

13.1.2 Context

A dating application has a connected data model: users, profile fields, media files, moderation records, search preferences, swipes, matches, chats, messages, subscriptions, transactions, dictionaries, and deletion snapshots. The feed also benefits from geospatial filtering and future recommendation support.

13.1.3 Decision

Use PostgreSQL as the primary relational database and enable PostGIS and pgvector. Laravel migrations define schema evolution. Eloquent models represent domain tables. The database is bootstrapped through Docker and `init.sql`.

13.1.4 Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
MySQL	Familiar relational database	Weaker fit for Post-GIS/pgvector combination	PostgreSQL supports both required extensions
MongoDB	Flexible documents	Harder relationships for swipes, chats, subscriptions, and transactions	Product data is relational

Alternative	Pros	Cons	Why Not Chosen
Separate vector database	Specialized vector search	Additional infrastructure too early	pgvector is enough for diploma scope

13.1.5 Consequences

Positive: - Strong relational integrity for dating-domain relationships. - Geospatial filtering is available inside the main database. - Vector-ready profile fields can be stored without adding another service. - Migrations document schema growth over time.

Negative: - PostgreSQL extensions must be present in every environment. - Some complex filters require careful indexing and query inspection.

Neutral: - The database is optimized for a single backend API, not multi-tenant SaaS isolation.

13.2 Implementation Details

13.2.1 Main Tables

users
user_feed_profile
user_search_preferences
user_files
user_moderation
user_swipes
chats
chat_users
chat_messages
countries
cities
activities
hobbies
subscriptions
subscription_items
transactions
telegram_subscriptions
user_deletion_snapshots

13.2.2 Key Implementation Decisions

Decision	Rationale
Separate feed profile table	Keeps dating profile/search fields separate from auth identity.
user_swipes table	Tracks decisions and excludes already-swiped profiles from feed.
Chat join table	Supports chat participants and leaves room for extension.
Dictionary tables	Countries, cities, activities, and hobbies are seeded and reusable.
Deletion snapshot table	Supports account deletion/restore business flow.

13.2.3 Diagrams

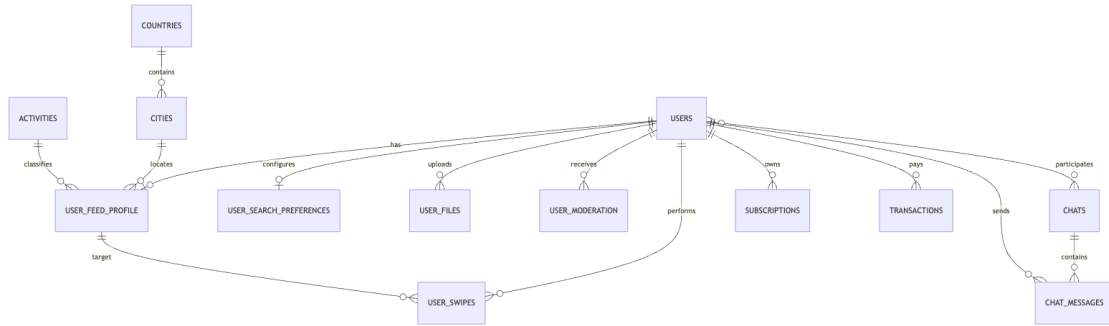


Figure 8: Diagram

13.3 Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Relational schema	Completed	Migrations define user, profile, feed, chat, payment, and dictionary tables.
2	ORM integration	Completed	Eloquent models exist under <code>api/app/Models</code> .
3	Geospatial support	Completed	PostGIS is enabled; feed can order by distance from city coordinates.
4	Vector support	Completed	pgvector extension and vector field migration exist.
5	Seed data	Completed	Seeders exist for countries/cities, activities, hobbies, and feed profiles.
6	Schema documentation	Completed	See Database Schema .

13.4 Known Limitations

Limitation	Impact	Potential Solution
Index strategy is not fully documented	Performance review needs extra work before production scale	Add indexes for common feed, chat, and moderation queries.
Full ERD generated from migrations is not committed	Diagram is maintained manually in docs	Generate schema diagrams from PostgreSQL metadata.

13.5 References

- `api/database/migrations/`
- `api/database/seaders/`
- `database/init.sql`
- `api/app/Models/`

14 Criterion: Real-Time Communication

14.1 Architecture Decision Record

14.1.1 Status

Status: Accepted

Date: May 3, 2026

14.1.2 Context

Dating chat and match feedback lose value when updates appear only after manual refresh. The application needs real-time signals for message sending, read states, chat creation, likes, matches, match deletion, and moderation/user updates.

14.1.3 Decision

Use Laravel events with broadcasting and Laravel Reverb for private channels. The frontend uses Laravel Echo-related dependencies and socket hooks to receive message and read events. Queue jobs and events keep user-facing updates separated from synchronous HTTP responses.

14.1.4 Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Polling	Simple and reliable	Slow, wasteful, poor chat UX	Chat needs immediate updates
Third-party realtime provider	Managed infrastructure	Extra cost and vendor dependency	Laravel Reverb is already in stack
Custom WebSocket server	Full control	More code and auth complexity	Reverb integrates with Laravel channels

14.1.5 Consequences

Positive: - Chat messages and read states can update without page refresh. - Private channels use Laravel authentication boundaries. - Events are explicit domain objects.

Negative: - Deployment must expose and configure socket host, port, and auth URL. - Local debugging requires both API and socket services.

Neutral: - Some notifications are also sent through Telegram bot jobs, which complements in-app realtime.

14.2 Implementation Details

14.2.1 Project Structure

```
api/app/Events/Chat/  
api/app/Events/Feed/  
api/app/Events/Moderation/  
api/routes/channels.php  
api/app/Broadcasting/
```

spa/src/shared/sockets/
spa/src/pages/chat/lib/

14.2.2 Key Implementation Decisions

Decision	Rationale
Domain-specific events	MessageSentEvent, MessageReadEvent, MatchEvent, and moderation events are easy to trace.
Private channel authorization	Chat and user updates must not be public.
Socket hooks in frontend	Keeps subscription logic reusable in chat pages and shared modules.

14.2.3 Diagrams

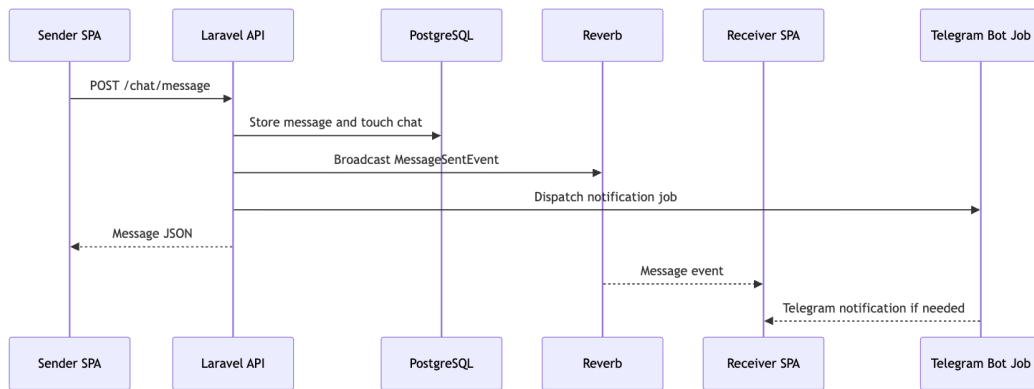


Figure 9: Diagram

14.3 Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Real-time events	Completed	Chat, feed, and moderation events exist under api/app/Events.
2	Private channels	Completed	Broadcasting channel classes and routes/channels.php exist.
3	Frontend subscriptions	Completed	Socket utilities and chat hooks exist in SPA.
4	Message read state	Completed	markMessageRead endpoint and MessageReadEvent exist.
5	Async notifications	Completed	Telegram notification jobs are dispatched for message events.

14.4 Known Limitations

Limitation	Impact	Potential Solution
Reconnect behavior needs production testing	Mobile network changes may interrupt sockets	Add explicit retry/backoff and UX indicators.
Load testing is not documented	Reverb capacity is unknown	Run socket load tests before production launch.

14.5 References

- `api/app/Events/Chat/`
- `api/app/Events/Feed/`
- `api/app/Broadcasting/`
- `spa/src/shared/sockets/`
- `spa/src/pages/chat/lib/useChatSocket.ts`

15 Criterion: Authentication and Telegram Integration

15.1 Architecture Decision Record

15.1.1 Status

Status: Accepted

Date: May 3, 2026

15.1.2 Context

The product is designed as a Telegram Mini App, so authentication should not imitate a separate email/password product. It should use Telegram as the user entry channel, support bot webhooks and notifications, and still protect backend routes with normal API authentication.

15.1.3 Decision

Use a Telegram login endpoint on the backend, Laravel Sanctum for authenticated API routes, Telegram bot webhook endpoints for user and moderation actions, and frontend Telegram SDK initialization for webview behavior.

15.1.4 Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Email/password login	Familiar	Adds registration friction and password handling	Telegram Mini App should use Telegram identity
OAuth provider login	Standard identity flow	Less integrated with Telegram bot and invoices	Telegram is the target platform

Alternative	Pros	Cons	Why Not Chosen
Anonymous sessions only	Fast initial access	Unsafe for dating, chat, media, and payments	User identity is required

15.1.5 Consequences

Positive: - Users start from a familiar Telegram context. - Bot notifications and invoices fit the same ecosystem. - Backend routes remain protected by Sanctum.

Negative: - Local testing requires Telegram test data or mocks. - Production correctness depends on securely validating Telegram login data and secrets.

Neutral: - The app is intentionally coupled to Telegram as a platform choice.

15.2 Implementation Details

15.2.1 Project Structure

```
api/app/Http/Controllers/Auth/TelegramController.php
api/app/Http/Controllers/Bot/TelegramBotController.php
api/app/Services/TelegramService.php
api/config/telegram.php
api/routes/api.php
spa/src/app/init.ts
spa/src/app/providers/AuthProvider.tsx
```

15.2.2 Key Implementation Decisions

Decision	Rationale
Public /login route	Telegram auth must be available before Sanctum-protected calls.
Protected route group	All user data and actions require authenticated identity.
Telegram bot webhooks	Bot and moderation events can enter the backend separately from SPA HTTP requests.
Frontend initialization	Telegram viewport, safe area, theme, closing behavior, location, and swipe behavior are configured in the SPA.

15.2.3 Diagrams

15.3 Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Platform authentication	Completed	POST /login handled by Telegram controller.
2	Protected API routes	Completed	Route group uses auth:sanctum.
3	Telegram webhooks	Completed	Bot and moderation webhook routes exist.

#	Requirement	Status	Evidence/Notes
4	Frontend Telegram SDK usage	Completed	Initialization is documented in SPA README and implemented in <code>src/app/init.ts</code> . Telegram service and notification jobs exist.
5	Bot notifications	Completed	

15.4 Known Limitations

Limitation	Impact	Potential Solution
Authentication test fixtures are not documented	Reviewers need manual Telegram setup details	Add mock login mode or documented test init data.
Platform coupling	Non-Telegram users cannot use full flow directly	Add optional email/social login only if product expands.

15.5 References

- `api/app/Http/Controllers/Auth/TelegramController.php`
- `api/app/Http/Controllers/Bot/TelegramBotController.php`
- `api/app/Services/TelegramService.php`
- `spa/src/app/init.ts`

16 Criterion: Media Storage and Moderation

16.1 Architecture Decision Record

16.1.1 Status

Status: Accepted

Date: May 3, 2026

16.1.2 Context

Dating applications rely on user photos and videos, but unsafe or invalid media can damage trust. The system needs upload validation, profile file records, moderation state, derived media processing, and frontend flows that prevent unresolved moderation problems from being ignored.

16.1.3 Decision

Implement media workflows through Laravel storage services, file upload request validation, file-specific rule classes, user file models, moderation records, moderation jobs, and dedicated frontend media/moderation screens. Local development uses MinIO, while the backend includes S3/GCS-compatible storage dependencies.

16.1.4 Alternatives Considered

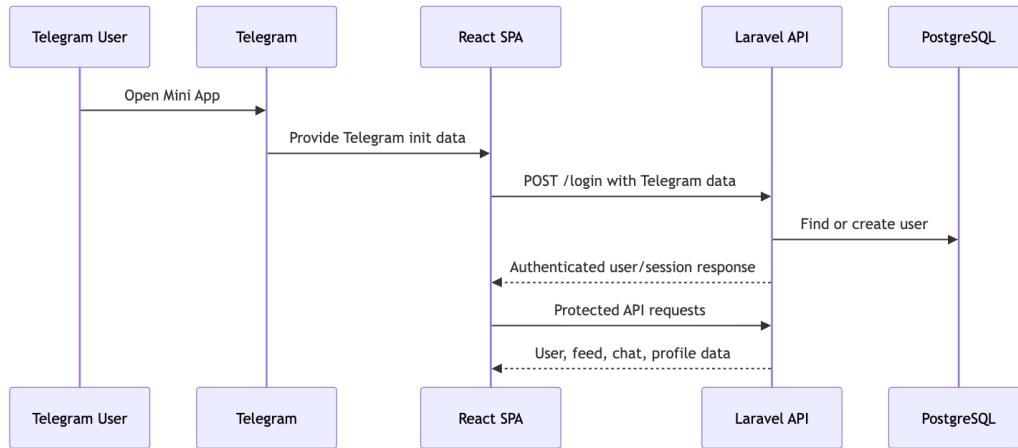


Figure 10: Diagram

Alternative	Pros	Cons	Why Not Chosen
Store files in database	Simple backup story	Poor performance and large database size	Object storage is better for media
Only client-side validation	Fast feedback	Not secure or trustworthy	Backend validation is required
Manual moderation only	Human judgment	Slow and incomplete for diploma scope	Automated jobs and moderation states are more practical

16.1.5 Consequences

Positive: - Upload and deletion are centralized. - Moderation status can control user routing. - Background jobs can process derived assets such as blurred photos. - Storage provider can change without rewriting controllers.

Negative: - Media workflows require object storage configuration. - Moderation decisions need careful UX so users understand blocked states.

Neutral: - Full human moderation tooling is outside current scope.

16.2 Implementation Details

16.2.1 Project Structure

```

api/app/Http/Controllers/Storage/FileController.php
api/app/Http/Controllers/Moderation/ModerationController.php
api/app/Http/Requests/Storage/
api/app/Rules/Storage/
api/app/Rules/UserProfile/
api/app/Services/User/FileService.php
api/app/Services/Storage/StorageService.php
api/app/Services/Moderation/ModerationService.php
api/app/Jobs/File/CreateBlurredPhotoJob.php

```

```

api/app/Jobs/Moderation/
spa/src/processes/moderation/
spa/src/shared/ui/FileInput/
spa/src/widgets/CameraModal/

```

16.2.2 Key Implementation Decisions

Decision	Rationale
Store metadata in user_files	Database tracks ownership, type, moderation, and main-photo state.
Use object storage	Scales better than filesystem/database media storage.
Moderate through jobs and records	Separates upload from moderation workflow and user routing.
Frontend route guard for moderation	Users with unresolved moderation cases are redirected before feed usage.

16.2.3 Diagrams

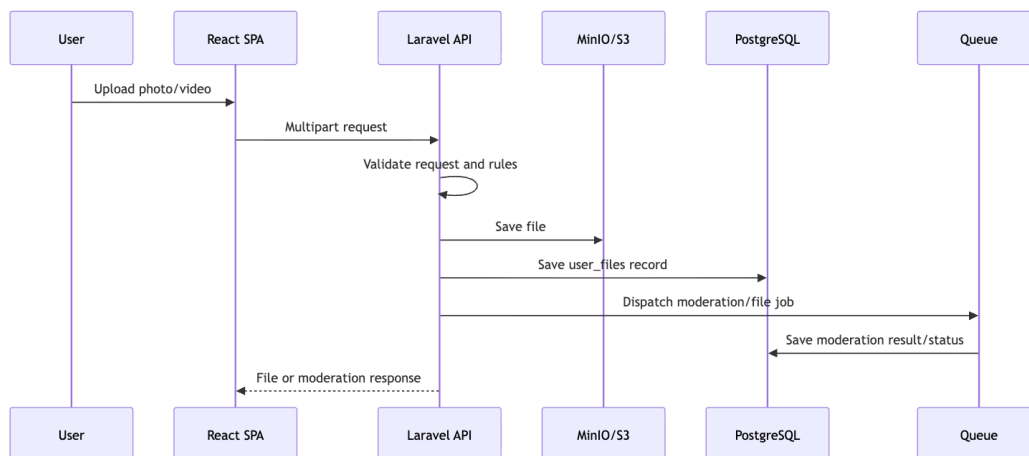


Figure 11: Diagram

16.3 Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Photo upload	Completed	Storage and user profile routes include photo upload/update.
2	Video upload	Completed	User and storage endpoints include video upload/delete.
3	Backend validation	Completed	Storage and user profile request/rule classes exist.
4	Moderation model	Completed	UserModeration model and migrations exist.
5	Background processing	Completed	Moderation and blurred photo jobs exist.

#	Requirement	Status	Evidence/Notes
6	Frontend moderation UX	Completed	Moderation process and route guard exist.

16.4 Known Limitations

Limitation	Impact	Potential Solution
Full moderator dashboard is missing	Manual review workflow is limited	Add admin moderation queue UI.
Media CDN strategy is not documented	Production media delivery may be slower	Add CDN/storage bucket configuration.

16.5 References

- `api/app/Services/Storage/StorageService.php`
- `api/app/Services/User/FileService.php`
- `api/app/Services/Moderation/ModerationService.php`
- `api/app/Jobs/Moderation/`
- `spa/src/processes/moderation/`

17 Criterion: Containerization and Deployment

17.1 Architecture Decision Record

17.1.1 Status

Status: Accepted

Date: May 3, 2026

17.1.2 Context

The application has multiple runtime dependencies: Laravel/PHP, React build tooling, PostgreSQL with extensions, Redis, MinIO, Nginx, and background workers. Manual installation would make review and deployment harder.

17.1.3 Decision

Provide Dockerfiles and Docker Compose files for the API, SPA, and database layer. The API compose stack includes Laravel, PostgreSQL/PostGIS/pgvector, Redis, and MinIO. The SPA has a Dockerfile and Nginx configuration for serving the production build. The database folder documents extension bootstrap separately.

17.1.4 Alternatives Considered

Alternative	Pros	Cons	Why Not Chosen
Manual local setup only	Simple files	Reviewers must install every service locally	Too fragile for diploma defense
Managed platform only	Production-like hosting	Less transparent and may require paid accounts	Local reproducibility is required
Kubernetes	Scalable	Excessive complexity for this scope	Docker Compose is enough

17.1.5 Consequences

Positive: - Reviewers can reproduce the backend support services. - Database extensions are explicit. - Frontend can be served as static files behind Nginx. - Local object storage is available through MinIO.

Negative: - Environment variables still need correct values. - Full CI/CD automation is documented but not fully committed as workflow files.

Neutral: - Compose files are appropriate for development and small deployments, not high-availability production.

17.2 Implementation Details

17.2.1 Project Structure

```
api/Dockerfile
api/docker-compose.yml
api/nginx.conf
api/docker/
api/docker/database/
spa/Dockerfile
spa/docker-compose.yml
spa/nginx.conf
database/docker-compose.yml
database/Dockerfile
database/init.sql
```

17.2.2 Key Implementation Decisions

Decision	Rationale
Separate API and SPA containers	Backend runtime and frontend static hosting have different needs.
PostGIS image/build	Database must include geospatial support.
Redis container	Queues and cache should not depend on local host installation.
MinIO container	Local media storage should behave like S3-compatible storage.

17.2.3 Diagrams

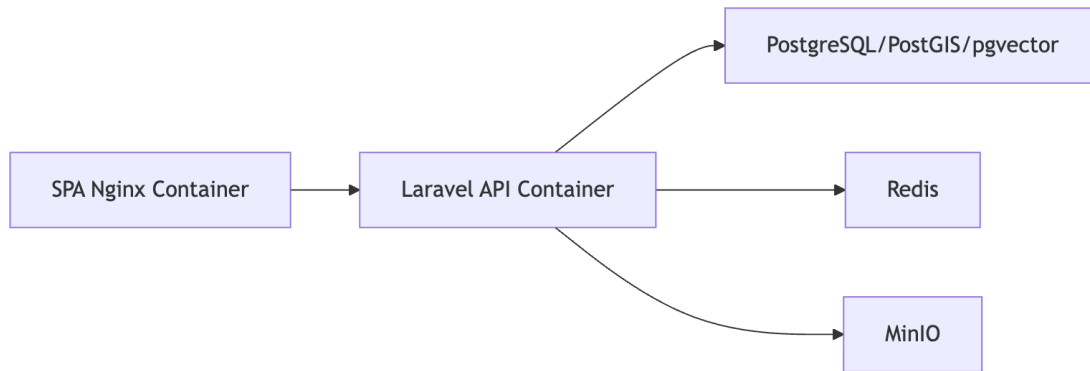


Figure 12: Diagram

17.3 Requirements Checklist

#	Requirement	Status	Evidence/Notes
1	Backend containerization	Completed	api/Dockerfile and api/docker-compose.yml.
2	Frontend containerization	Completed	spa/Dockerfile, spa/docker-compose.yml, and spa/nginx.conf.
3	Database containerization	Completed	database/docker-compose.yml, database/Dockerfile, database/init.sql.
4	Supporting services	Completed	Redis and MinIO are in backend compose stack.
5	Local run documentation	Completed	README files and deployment page describe commands.
6	CI/CD	Partially completed	Expected pipeline is documented; workflow file is not committed.

17.4 Known Limitations

Limitation	Impact	Potential Solution
No committed production CI workflow	Deployment remains manual	Add GitHub Actions or GitLab CI file.
Production URLs not specified	Final submission needs updated links	Add production environment details after deployment.

17.5 References

- api/docker-compose.yml
- spa/docker-compose.yml
- database/README.md
- thesis-template/docs/02-technical/deployment.md

18 3. User Guide

This section explains how an end user works with PlusDateApp. The application is intended primarily for Telegram Mini App usage on mobile devices.

18.1 Contents

- [Features Walkthrough](#)
- [FAQ & Troubleshooting](#)

18.2 Getting Started

18.2.1 System Requirements

Requirement	Minimum	Recommended
Browser	Current Telegram mobile webview or modern Chrome/Safari	Latest Telegram app and latest mobile OS browser engine
Screen Resolution	Mobile viewport supported by Telegram webview	Modern smartphone screen
Internet Device	Required Mobile-first; desktop browser can be used for development	Stable mobile or Wi-Fi connection Telegram mobile app

18.2.2 Accessing the Application

1. Open the PlusDate Telegram bot or Mini App link.
2. Launch the Mini App from Telegram.
3. Allow Telegram to pass the Mini App initialization data.
4. Wait for the application to authenticate and load the current user.

For local development, start the backend and frontend from the commands in [Deployment & DevOps](#), then open the Vite URL in a browser or Telegram-compatible test environment.

18.2.3 First Launch

18.2.3.1 Step 1: Telegram Login

1. The user opens the Mini App inside Telegram.
2. The frontend initializes Telegram webview behavior.
3. The frontend sends Telegram login data to the backend.
4. The backend returns the authenticated user state.

18.2.3.2 Step 2: Onboarding

1. Fill in basic profile data.
2. Select age and city.
3. Add interests and activity information.
4. Upload profile media.
5. Complete verification media if required.
6. Submit the profile for moderation.

18.2.3.3 Step 3: Moderation After onboarding, the system checks media and profile state. If there is an unresolved moderation issue, the user is redirected to the moderation screen and cannot use the normal feed until the issue is resolved.

18.2.3.4 Step 4: Main Feed After onboarding and moderation requirements are satisfied, the user is redirected to the feed. The footer navigation gives access to feed, likes, chats, and profile.

18.3 Quick Start Guide

Task	How To
Create a profile	Open the Mini App, follow onboarding, add required fields and media.
Change search preferences	Open profile/preferences screens and update city, activity, interests, eye color, or other filters.
Browse profiles	Open Feed and swipe through profile cards.
Like someone	Use the like action on a feed card.
Respond to a like	Open Likes and accept or reject the incoming like.
Chat with a match	Open Chats, select a conversation, and send a message.
Buy premium	Open Premium and choose the available subscription/payment flow.
Delete account	Open Profile Settings and follow the delete account flow.

18.4 User Roles

Role	Permissions	Access Level
Guest Telegram user	Can open the Mini App and authenticate.	Limited until onboarding is complete
Onboarded user	Can edit profile, use feed, likes, matches, chat, and settings.	Standard
Premium user	Can use premium-facing functions and subscription benefits.	Extended
Admin/moderator	Can use admin endpoints such as bot profile creation and user deletion.	Operational

19 Feature Walkthrough

19.1 Feature 1: Onboarding and Profile Creation

19.1.1 Overview

Onboarding collects the minimum data needed before a user appears in discovery: basic details, age, city, interests, media, and verification. The RouterGuard prevents users from skipping required steps.

19.1.2 How to Use

Step 1: Open the Mini App from Telegram.

Step 2: Complete the basic profile screen.

Step 3: Select age and city.

Step 4: Choose interests and activity.

Step 5: Upload photos, video if available, and verification media.

Expected Result: The user profile is stored, onboarding status is updated, and the user moves to moderation or feed depending on profile state.

19.1.3 Tips

- Use clear profile photos to reduce moderation issues.
 - Complete search preferences early because the feed depends on them.
-

19.2 Feature 2: Feed and Swiping

19.2.1 Overview

The feed shows profile cards that match the user's preferences and excludes profiles that have already been swiped. Users can like, dislike, and revert selected actions when allowed.

19.2.2 How to Use

Step 1: Open the Feed tab.

Step 2: Review the current profile card, photos, and profile details.

Step 3: Use the swipe actions to like or dislike.

Step 4: If a mutual like happens, the match flow appears and the chat can be opened.

Expected Result: The decision is stored by the backend and the next profile appears. If a match is created, the user receives match feedback.

19.3 Feature 3: Likes and Matches

19.3.1 Overview

The Likes page lets users review people who liked them and decide whether to respond. This connects discovery to chat creation.

19.3.2 How to Use

Step 1: Open the Likes tab from the footer.

Step 2: Review the incoming like card.

Step 3: Accept the like to create a match or reject it to remove the candidate.

Expected Result: The backend stores the response and creates a match when both users like each other.

19.4 Feature 4: Chat

19.4.1 Overview

Chat is available for matched users. It includes chat list, message history, message sending, unread counts, and read state updates.

19.4.2 How to Use

Step 1: Open the Chats tab.

Step 2: Select a conversation.

Step 3: Type a message and send it.

Step 4: Read incoming messages; read state updates are sent back to the backend.

Expected Result: Messages are stored in the database and real-time events update the other user's interface.

19.5 Feature 5: Profile Editing and Preferences

19.5.1 Overview

Users can update their public profile and search settings after onboarding. The frontend provides separate screens for editing city, age, height, eye color, interests, activity, and search preferences.

19.5.2 How to Use

Step 1: Open the Profile tab.

Step 2: Choose the profile field or settings area to edit.

Step 3: Save the updated value.

Expected Result: The backend validates and stores the updated profile or preference data. The feed uses the new preferences on future requests.

19.6 Feature 6: Premium and Payments

19.6.1 Overview

Premium screens present paid features and connect to subscription endpoints. The backend supports current subscription, subscribe, cancel, Stripe Cashier, and Telegram invoice flows.

19.6.2 How to Use

Step 1: Open the Premium page from the profile or action limit modal.

Step 2: Review premium benefits.

Step 3: Start the subscription or Telegram invoice flow.

Expected Result: The backend creates or updates subscription state and the frontend shows payment success or error feedback.

19.7 Feature Comparison

Feature	Standard User	Premium User
Onboarding and profile editing	Available	Available
Feed and basic swipes	Available with limits	Extended or less restricted

Feature	Standard User	Premium User
Likes/matches	Available	Enhanced visibility depending on premium rules
Revert dislike	Limited or gated	Available where allowed
Advanced filters	Limited	Available where allowed
Chat after match	Available	Available

20 FAQ & Troubleshooting

20.1 Frequently Asked Questions

20.1.1 General

Q: What is PlusDateApp?

A: PlusDateApp is a Telegram Mini App dating platform with profile onboarding, media moderation, discovery feed, swipes, likes, matches, chat, and premium subscription flows.

Q: Do I need a separate account?

A: No separate email/password account is planned for the main flow. The application uses Telegram as the entry point and the backend protects user routes through Laravel Sanctum.

Q: Can I use the app outside Telegram?

A: Developers can run the SPA in a browser for local testing, but the intended product experience depends on Telegram Mini App initialization.

20.1.2 Account & Access

Q: Why am I redirected to onboarding?

A: The account is not fully onboarded. Complete all required onboarding steps before using the feed.

Q: Why am I redirected to moderation?

A: Your profile has an unresolved moderation issue, usually related to uploaded media or verification requirements.

Q: Can I delete my account?

A: Yes. The profile/settings flow includes account deletion, and the backend stores deletion snapshot data for restore-related behavior.

20.1.3 Features

Q: Why do I see no profiles in the feed?

A: Possible reasons include strict search preferences, no available profiles in seed data, unresolved moderation, or all matching profiles already being swiped.

Q: When can I chat with another user?

A: Chat is intended for matched users. A match is created when both users express a positive decision.

Q: What does premium provide?

A: Premium is represented by the premium page, subscription endpoints, and premium-facing UI such as limits, filters, likes, and revert-related flows. Exact benefits should be finalized in the deployed product copy.

20.2 Troubleshooting

20.2.1 Common Issues

Problem	Possible Cause	Solution
Page does not load locally API requests fail locally	Frontend dev server is not running Backend or Docker services are stopped	Run <code>npm run dev</code> in spa. Start API services and check <code>/api/healthcheck</code> .
Login fails	Telegram initialization data is missing or invalid	Test inside Telegram Mini App context or add a documented local auth mock.
Feed is empty	No seeded profiles, strict preferences, or moderation state	Seed database and relax preferences.
Upload fails	File type, size, storage, or credentials issue	Check storage environment variables and backend logs.
Chat does not update live	Socket host/auth URL is misconfigured	Check <code>VITE_SOCKET_*</code> variables and Laravel Reverb configuration.
Payment flow fails	Stripe or Telegram invoice credentials are not configured	Check payment environment variables and backend logs.

20.2.2 Error Messages

Error Code/Message	Meaning	How to Fix
401 Unauthorized	User is not authenticated or Sanctum session/token is missing.	Log in through Telegram again and check API auth configuration.
403 Forbidden	The action is blocked by a rule or policy.	Verify profile state, moderation state, and ownership of the resource.
422 Unprocessable Entity	Request validation failed.	Check required fields, file format, and form values.

Error Code/Message	Meaning	How to Fix
500 Internal Server Error	Backend exception or missing infrastructure dependency.	Check Laravel logs, .env, database, Redis, and storage services.

20.2.3 Browser-Specific Issues

Browser	Known Issue	Workaround
Telegram WebView	Depends on Telegram version and platform behavior.	Use the latest Telegram app.
Safari iOS	Media upload permissions may require user interaction.	Reopen upload dialog and check browser permissions.
Desktop Chrome	Telegram-specific APIs may be absent.	Use desktop only for development, not final product testing.

20.3 Getting Help

20.3.1 Self-Service Resources

- [Project Overview](#)
- [Deployment & DevOps](#)
- [API Reference](#)
- [Database Schema](#)

20.3.2 Contact Support

Channel	Response Time	Best For
Repository issue tracker	Depends on project owner	Bugs, technical improvements, and documentation fixes
Supervisor communication	Academic schedule	Diploma scope and assessment questions

20.3.3 Reporting Bugs

When reporting a bug, include:

1. Steps to reproduce.
2. Expected behavior.
3. Actual behavior.
4. Screenshot or screen recording if relevant.
5. Browser, device, Telegram version, and environment.
6. Backend logs or network response if available.

21 4. Retrospective

This section reflects on the project development process, technical results, known issues, and future improvements.

21.1 What Went Well

21.1.1 Technical Successes

- The project reached a complete full-stack shape instead of remaining a prototype: backend, frontend, database, storage, queues, sockets, and Docker assets are present.
- Laravel worked well for a domain-heavy API because requests, rules, services, events, jobs, resources, migrations, and queues could be organized by product flow.
- React with TypeScript and Vite supported a rich Telegram Mini App interface with many screens and shared UI components.
- PostgreSQL with PostGIS and pgvector gave the data layer room for location-based and vector-ready discovery features.
- The chat service uses transactions and advisory locks to avoid duplicate chats between the same users.
- The repository includes README files and documentation sources that make review easier.

21.1.2 Process Successes

- Splitting the repository into api, spa, and database made responsibilities clear.
- The feature-sliced frontend structure prevented all UI code from being concentrated in a single pages folder.
- The backend service layer kept business logic more understandable than fat controllers.
- Docker assets reduced the amount of manual infrastructure setup needed for review.

21.1.3 Personal Achievements

- Built and documented a multi-module product instead of a single isolated application.
- Practiced integrating external services such as Telegram, Stripe, Sentry, storage providers, and Reverb.
- Improved understanding of real product constraints: onboarding gates, moderation, media validation, profile visibility, and account lifecycle.

21.2 What Did Not Go As Planned

Planned	Actual Outcome	Cause	Impact
Complete production deployment links	Production is available through Telegram bot @stage_plusdate_bot, but a separate public web URL is not specified.	Product is distributed through Telegram Mini App.	Low
Generated API documentation	API reference is manually documented.	OpenAPI annotations/generator were not added during implementation.	Medium
Full automated UI tests	Backend has some feature tests, but frontend E2E tests are absent.	Time was spent on product flows and integrations.	Medium
Complete admin/moderation dashboard	Backend admin/moderation endpoints exist, but full dashboard is out of scope.	Diploma scope prioritized user-facing flows.	Low

21.2.1 Challenges Encountered

1. Telegram-specific local testing

- Problem: Some frontend behavior depends on Telegram webview APIs.
- Impact: Browser-only development cannot fully represent production behavior.
- Resolution: Telegram initialization was isolated in app bootstrap and documented as a platform dependency.

2. Media and moderation complexity

- Problem: Uploading files is not only a storage task; it affects trust, profile visibility, and user routing.
- Impact: More backend rules, jobs, and frontend states were required.
- Resolution: Media, moderation, and route-guard behavior were separated into services, jobs, and dedicated screens.

3. Real-time configuration

- Problem: Chat requires HTTP API, private channel authorization, socket host variables, and frontend subscriptions to align.
- Impact: Deployment has more environment variables and test cases.
- Resolution: Reverb/Echo-related configuration was kept explicit and documented.

21.3 Technical Debt & Known Issues

ID	Issue	Severity	Description	Potential Fix
TD-001	Limited production URL details	Low	Documentation includes Telegram bot @stage_plusdate_bot, but not a separate API or web hosting URL.	Add API/web URLs if they are exposed separately.
TD-002	Manual API docs	Medium	API reference can become outdated as routes change.	Add OpenAPI generation or route-to-doc tooling.
TD-003	Limited automated coverage	Medium	Chat, moderation, storage, payments, and frontend flows need more tests.	Add PHPUnit feature tests and Playwright E2E tests.
TD-004	No full moderator dashboard	Low	Moderation exists, but operations UI is limited.	Build admin moderation queue and review screens.
TD-005	Performance indexes not fully documented	Low	Feed and chat queries may need optimization at scale.	Add query plans, indexes, and load testing notes.

21.3.1 Code Quality Issues

- API documentation should be generated from code or route metadata.
- Frontend should add E2E tests for onboarding, feed, likes, and chat.
- Storage and payment flows need documented mock/stub modes for reviewers without real credentials.

21.4 Future Improvements

21.4.1 High Priority

1. OpenAPI documentation

- Description: Generate API docs from route/controller annotations or a maintained OpenAPI file.
- Value: Reduces mismatch between implementation and documentation.
- Effort: Medium.

2. End-to-end test suite

- Description: Add Playwright tests for onboarding, feed, chat, premium, and moderation.
- Value: Protects the most important user workflows.
- Effort: Medium to high.

21.4.2 Medium Priority

3. Moderator dashboard

- Description: Build a web UI for reviewing media and moderation records.
- Value: Makes safety operations more realistic.

4. Recommendation improvements

- Description: Use the existing vector-ready database support for profile similarity and better ranking.
- Value: Improves feed relevance.

21.4.3 Nice to Have

5. Add a formal production CI/CD workflow.
6. Add CDN configuration for media delivery.
7. Add analytics dashboards for onboarding drop-off, swipes, matches, and subscription conversion.

21.5 Lessons Learned

21.5.1 Technical Lessons

Lesson	Context	Application
Product workflows should drive architecture	Dating apps require connected onboarding, moderation, feed, chat, and payments.	Keep domain services aligned with user journeys.
Real-time chat is more than sockets	Auth, persistence, unread state, events, and notifications all interact.	Design message flow end-to-end before coding UI.
Media upload affects trust	Photos/videos influence safety and user confidence.	Treat storage, validation, moderation, and routing as one workflow.

21.5.2 Process Lessons

Lesson	Context	Application
Documentation should be written progressively	Waiting until the end makes it harder to remember decisions.	Keep ADRs close to implementation work.
Local reproducibility matters	Reviewers need to run services without guessing dependencies.	Maintain Docker and README files as part of the product.

21.5.3 What Would Be Done Differently

Area	Current Approach	What Would Change	Why
Planning	Build many flows first, document at the end.	Write ADR notes during each major decision.	Reduces final documentation effort.
Technology	Manual API reference.	Add OpenAPI early.	Keeps docs accurate.
Process	Limited frontend automated tests.	Add E2E tests as soon as onboarding/feed stabilize.	Prevents regressions in complex UI flows.
Scope	Admin dashboard out of scope.	Define a small moderation dashboard earlier.	Makes safety operations more demonstrable.

21.6 Personal Growth

21.6.1 Skills Developed

Skill	Before Project	After Project
Full-stack architecture	Intermediate	Stronger understanding of service boundaries and frontend structure
Laravel backend development	Intermediate	Stronger use of requests, services, DTOs, jobs, events, and migrations
React SPA development	Intermediate	Stronger use of routing, state, server queries, shared UI, and Telegram-specific UX
DevOps basics	Basic to intermediate	Better Docker, service dependency, and deployment documentation skills

21.6.2 Key Takeaways

1. A diploma application should demonstrate complete product workflows, not only isolated technical features.
2. Good documentation explains why decisions were made, not only what libraries were installed.
3. Moderation, payments, storage, and real-time communication introduce product-level complexity that must be handled deliberately.

Retrospective completed: May 3, 2026

22 5. Appendices

Supplementary reference materials for PlusDateApp.

22.1 Contents

- [Glossary](#) - key product and technical terms.
- [API Reference](#) - main API endpoint groups from `api/routes/api.php`.
- [Database Schema](#) - main tables, relationships, migrations, and seeding.

23 Glossary

Term	Definition
PlusDateApp	Telegram Mini App dating platform implemented in this diploma project.
Mini App	A web application opened inside Telegram's webview.
Onboarding	Guided setup flow that collects required profile data before feed access.
Feed	Discovery screen that shows profile cards for swiping.
Swipe	User decision on a profile, such as like or dislike.
Match	Mutual positive decision between two users that enables conversation.
Moderation	Process of validating profile media or user state before normal access.
Premium	Paid access tier represented by subscription and premium UI flows.
Dictionary	Reference data such as countries, cities, activities, and hobbies.
Deletion snapshot	Stored data used by account deletion/restore workflow.

23.1 Acronyms

Acronym	Full Form	Description
API	Application Programming Interface	Backend endpoints used by the SPA.
SPA	Single Page Application	Frontend application rendered in the browser/webview.
DTO	Data Transfer Object	Structured object used to pass data between layers.
ORM	Object-Relational Mapper	Eloquent maps PHP models to database tables.
ADR	Architecture Decision Record	Documentation format used for evaluation criteria.
GTM	Google Tag Manager	Analytics integration used by the frontend.
CDN	Content Delivery Network	Potential future media delivery layer.

23.2 Domain-Specific Terms

23.2.1 Dating Domain

Term	Definition
Search preferences	User settings that control which profiles appear in the feed.

Term	Definition
Incoming like	A positive decision made by another user before the current user responds.
Revert swipe	Premium-style action that undoes a previous swipe decision when allowed.
Verification media	Photo or video used to increase trust and support moderation.

23.2.2 Technical Domain

Term	Definition
PostGIS	PostgreSQL extension used for geospatial data and distance calculations.
pgvector	PostgreSQL extension used for vector fields and similarity-oriented data.
Reverb	Laravel realtime server used for broadcasting events.
Sanctum	Laravel package used to protect authenticated API routes.
MinIO	Local S3-compatible object storage used for media in development.

24 API Reference

24.1 Overview

Base URL for local development: `http://localhost:8000/api`

Authentication: Telegram login followed by Laravel Sanctum-protected requests. Public routes are limited to login, Telegram webhooks, moderation webhook, and healthcheck.

24.2 Public Endpoints

Method	Path	Purpose
POST	<code>/login</code>	Authenticate or create a user from Telegram login data.
POST	<code>/telegram/webhook</code>	Receive Telegram bot webhook events.
POST	<code>/telegram/moderation/webhook</code>	Receive Telegram moderation webhook events.
GET	<code>/healthcheck</code>	Return API health status.

Example healthcheck response:

```
{
  "status": "ok"
}
```

24.3 Authenticated Endpoint Groups

All endpoints below are inside the `auth:sanctum` and `RecordActivityMiddleware` route group.

24.3.1 Current User

Method	Path	Purpose
GET	/me	Return the authenticated user profile state.

24.3.2 Moderation

Method	Path	Purpose
GET	/moderation/user-profile/photos	Validate current user's profile photos.

24.3.3 Dictionary

Method	Path	Purpose
GET	/dictionary/cities	List cities.
GET	/dictionary/countries	List countries.
GET	/dictionary/activities	List activities.
GET	/dictionary/hobbies	List hobbies.

24.3.4 User Profile

Method	Path	Purpose
GET	/user/{id}	Get another user's profile by numeric ID.
POST	/user/profile	Create or update profile information.
PUT	/user/search/preferences	Update search preferences.
GET	/user/likes	Get incoming likes.
POST	/user/photos	Update profile photos.
POST	/user/photo/{photoId}/main	Set main profile photo.
GET	/user/files	List current user's files.
POST	/user/files	Update files with moderation workflow.
POST	/user/files/video	Upload user video.
DELETE	/user/files/video/{id}	Delete user video.
GET	/user/swipes	Get available swipes or limits.
GET	/user/stats	Get user statistics.
POST	/user/onboard	Complete onboarding.
DELETE	/user/account	Delete account.

24.3.5 Feed

Method	Path	Purpose
POST	/feed/swipe	Store a like/dislike decision.
POST	/feed/swipe/revert	Revert a previous swipe when allowed.
GET	/feed/profiles	Get feed profiles with cursor pagination and filters.
DELETE	/feed/match	Delete an existing match.
POST	/likes/respond	Accept or reject an incoming like.

24.3.6 Chat

Method	Path	Purpose
POST	/chat	Create or get chat between two users.
GET	/chat	List current user's chats.
GET	/chat/{chat_id}/message	Get chat messages with cursor pagination.
POST	/chat/message	Send message.
PUT	/chat/message	Mark message as read.
GET	/chat/recent	Get recent chats.

24.3.7 Storage

Method	Path	Purpose
POST	/storage/file/photo	Upload photo file.
POST	/storage/file/video	Upload video file.
DELETE	/storage/file	Delete stored file.

24.3.8 Payment

Method	Path	Purpose
GET	/payment/subscription	Get current subscription.
POST	/payment/subscription/cancel	Cancel subscription.
POST	/payment/subscribe	Start subscription flow.
POST	/payment/telegram/invoice	Send Telegram invoice.

24.3.9 Admin

Method	Path	Purpose
POST	/admin/create-bot	Create a bot/test profile.
DELETE	/admin/user/{user_id}	Delete user account as admin.

24.4 Common Error Responses

Status Code	Meaning	Typical Cause
200	Success	Request completed.
201	Created	Resource was created.
401	Unauthorized	Missing or invalid authentication.
403	Forbidden	Domain rule blocks the action.
404	Not Found	Resource does not exist or is not visible to user.
422	Validation Error	Request body or file failed validation.
500	Server Error	Unexpected backend error or missing service configuration.

24.5 OpenAPI Status

There is no generated Swagger/OpenAPI file committed in the repository. This reference is based on `api/routes/api.php` and should be replaced or supplemented with generated OpenAPI documentation before production release.

25 Database Schema

25.1 Overview

Attribute	Value
Database	PostgreSQL with PostGIS and pgvector
Version	PostgreSQL 15 image in Docker setup
ORM	Laravel Eloquent
Schema Management	Laravel migrations

25.2 Entity Relationship Diagram

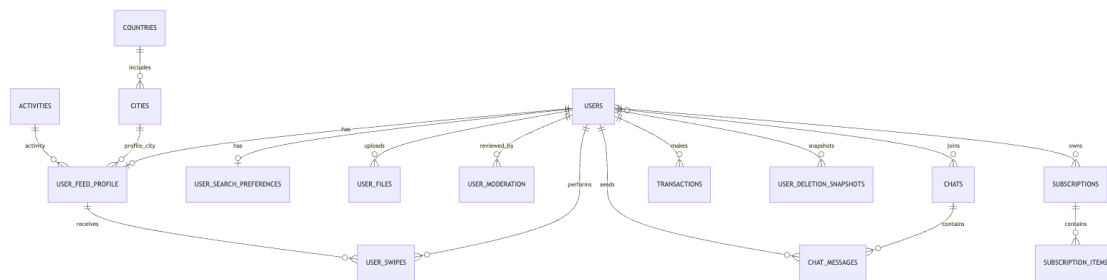


Figure 13: Diagram

25.3 Tables

25.3.1 users

Stores the authenticated user identity and account-level flags.

Column Group	Description
Identity	Telegram/user identity, name-related fields, contact/payment customer fields.
State flags	Onboarded, premium, under moderation, soft delete fields.
Timestamps	Standard creation/update/delete timestamps.

25.3.2 user_feed_profile

Stores public dating profile and feed-searchable fields.

Column Group	Description
Profile basics	Age, sex, height, activity, city, interests/hobbies, eye color, search purpose.
Location/vector	Coordinates and vector-ready field for advanced discovery.
Ownership	Linked to users.

25.3.3 user_search_preferences

Stores the user's desired feed filters.

Column Group	Description
Demographics	Gender, age range, height range, eye color.
Discovery filters	City, activity, hobbies, premium/video flags, search purpose.
Ownership	Linked to users.

25.3.4 user_files

Stores metadata for uploaded profile media.

Column Group	Description
File metadata	Type, path/provider metadata, moderation state, main-photo flag.
Ownership	Linked to users.
Processing	Used by media jobs such as blurred photo creation.

25.3.5 user_moderation

Stores moderation records and rejection/resolution state for user media/profile issues.

Column Group	Description
Moderation result	Rejection reason, resolution flag, related file/profile data.
Ownership	Linked to users.

25.3.6 user_swipes

Stores user decisions in the feed.

Column Group	Description
Decision	Swipe action and match-related state.
Relationships	Links swiping user and target user_feed_profile.

25.3.7 chats, chat_users, chat_messages

Stores one-to-one conversations, participants, messages, unread/read state, and timestamps.

Table	Purpose
chats	Conversation container.
chat_users	Participants and join metadata.
chat_messages	Message content, sender, sent time, and read time.

25.3.8 dictionaries

Stores reusable reference values.

Table	Purpose
countries	Country names and metadata.
cities	City names, country relation, and location data.
activities	User activity/profession categories with optional emoji.
hobbies	Interest/hobby reference data.

25.3.9 subscriptions and transactions

Stores premium and payment data.

Table	Purpose
subscriptions	Laravel Cashier subscription records.
subscription_items	Subscription item records.
transactions	Payment transaction data.
telegram_subscriptions	Telegram-specific subscription or invoice state.

25.3.10 account lifecycle tables

Table	Purpose
user_deletion_snapshots	Account deletion restore-related data.
user_bot_notifications	Bot notification state.
reflinks	Referral link-related data.
start_log	Bot/start tracking data.
personal_access_tokens	Sanctum token storage.
failed_jobs	Laravel failed queue job records.

25.4 Relationships

Relationship	Type	Description
users to user_feed_profile	One-to-One	A user has one public feed profile.
users to user_search_preferences	One-to-One	A user has one preference set for feed filtering.
users to user_files	One-to-Many	A user uploads multiple media files.

Relationship	Type	Description
users to user_swipes	One-to-Many	A user performs multiple feed decisions.
user_feed_profile to user_swipes	One-to-Many	A profile can receive many swipe decisions.
users to chats	Many-to-Many	Users participate in chats through chat_users.
chats to chat_messages	One-to-Many	A chat contains many messages.
countries to cities	One-to-Many	A country contains many cities.
cities to user_feed_profile	One-to-Many	Many profiles can be located in one city.
users to subscriptions	One-to-Many	A user may have subscription records.

25.5 Migrations

Area	Example Migration Files
Users/Auth	0001_01_01_000000_create_users_table.php, 2025_07_21_183612_create_personal_access_tokens_table.php
Dictionaries	2025_07_24_150607_create_countries_table.php, 2025_07_24_150617_create_cities_table.php, 2025_07_25_140759_create_activities_table.php, 2025_07_28_183037_create_hobbies_table.php
Profiles	2025_07_24_185455_create_user_feed_profile_table.php, 2025_08_17_115520_create_user_search_preferences_table.php
Media/Moderation	2025_08_08_144311_create_user_files_table.php, 2025_08_13_135638_create_user_moderation_table.php
Feed/Chat	2025_09_02_182822_create_user_swipes_table.php, 2025_08_24_122303_create_chats_table.php, 2025_08_24_122401_create_chat_users_table.php, 2025_08_24_122405_create_chat_messages_table.php
Payments	2025_09_10_150750_create_customer_columns.php, 2025_09_10_150751_create_subscriptions_table.php, 2025_09_10_150752_create_subscription_items_table.php, 2025_11_27_083518_create_transactions_table.php
Account lifecycle	2025_12_16_115154_create_user_deletion_snapshots_table.php

25.6 Seeding

Seeders are available for core dictionary and test data:

```
cd api
php artisan migrate --seed
```

Important seeders include:

- CountriesAndCitiesSeeder
- ActivitySeeder
- HobbiesSeeder
- UserFeedProfileSeeder
- DatabaseSeeder